

Instituto Tecnológico Superior de Xalapa

Subdirección de Posgrado e Investigación

EasyCountCol: Software para el conteo de unidades
formadoras de colonias de microorganismos a
través de visión artificial

Propuesta de Investigación

Que presenta:

Jiménez Villa Luis Alberto

Para la obtención del grado de

Maestro en Sistemas Computacionales

Directora: Mtra. María Salomé Alejandre Apolinar

Co-Directora: Dra. Virginia Lagunes Barradas

7 de diciembre del 2025

DEDICATORIA

AGRADECIMIENTOS

CONTENIDO

CAPÍTULO I. GENERALIDADES DEL PROYECTO	10
1.1 ANTECEDENTES	10
1.2 IDENTIFICACIÓN DEL PROBLEMA	17
1.3 OBJETIVOS	19
1.3.1 Objetivo general	19
1.3.2 Objetivos específicos	19
1.4 JUSTIFICACIÓN	20
1.5 ALCANCES Y LIMITACIONES	22
1.5.1 Alcances	22
1.5.2 Limitaciones	23
1.6 PREGUNTA DE INVESTIGACIÓN	24
1.6.1 Preguntas específicas	24
1.7 HIPÓTESIS	25
1.8 RIESGOS Y CONSIDERACIONES ÉTICAS	26
1.8.1 Riesgos	26
1.8.2 Consideraciones éticas	27
CAPÍTULO II. MARCO TEÓRICO	28
2.1 Fundamentos microbiológicos	28
2.1.1 Introducción a la microbiología	29
2.1.2 Aplicaciones de la microbiología en sectores clave	29
2.1.2.1 Sector clínico	29
2.1.2.2 Industria alimentaria	30
2.1.2.3 Industria farmacéutica	30
2.1.2.4 Medio ambiente	30
2.1.3 Microorganismos	31
2.1.3.1 Clasificación de los microorganismos	31
2.1.3.2 Morfología de las colonias microbianas	32
2.1.4 Unidades Formadoras de Colonias (UFC)	33
2.1.4.1 Principios básicos	33
2.1.4.2 Parámetros para un conteo válido	34
2.1.4.3 Expresión de resultados	34
2.1.5 Métodos tradicionales para el conteo microbiano	36
2.1.5.1 Técnica de cultivo en caja Petri	37
2.1.5.2 Medios de cultivos más utilizados	38
2.1.5.3 Limitaciones del conteo manual	40
2.2 Procesamiento digital de imágenes	40
2.2.1 Aplicaciones del procesamiento digital de imágenes	40
2.2.2 Etapas para el análisis de una imagen	41
2.2.3 Parámetros de una imagen	43
2.2.3.1 Resolución espacial	44
2.2.3.2 Resolución radiométrica	46
2.2.3.3 Modelos de color	47
2.2.3.4 Enfoque	48
2.2.3.5 Ruido	49
2.2.3.6 Contraste	49
2.2.3.7 Relación de aspecto	50
2.2.4 Métricas de evaluación de calidad de una imagen	51
2.2.4.1 Relación señal-ruido (SNR)	52

2.3. Visión artificial	52
2.3.1 Fundamentos de visión artificial	52
2.3.2 Principales aplicaciones de la visión artificial	53
2.3.3. Técnicas comunes en visión artificial	53
2.3.3.1 Aprendizaje automático (ML).....	53
2.3.3.2 Redes Neuronales Convolucionales (CNN)	55
2.3.3.3 Redes Generativas (GANs)	55
2.3.3.4 Transformers (ViT).....	56
2.3.4. Métricas de rendimiento.....	56
2.3.4.1 Intersección sobre la unión	57
2.3.4.2 Matriz de confusión	58
2.3.4.3 Precisión.....	59
2.3.4.4 Recall	60
2.3.4.5 Accuracy.....	60
2.3.4.6 Precisión media (mAP)	61
2.4 Tecnologías utilizadas en EasyCountCol	61
2.4.1 Lenguajes de programación.....	61
2.4.1.1 Python.....	62
2.4.1.2 Dart	62
2.4.1.3 SQL.....	63
2.4.2 Bibliotecas y frameworks de visión artificial.....	63
2.4.2.1 TensorFlow	63
2.4.2.2 PyTorch	63
2.4.2.3 YOLO	64
2.4.2.4 OpenCV.....	65
2.4.2.5 Pillow.....	65
2.4.3 Otras herramientas tecnológicas	66
2.4.3.1 Flutter	66
2.4.3.2 Sistemas operativos en dispositivos móviles	67
2.4.3.2.1 Android	67
2.4.3.2.2 IOS	68
2.4.4.1 Git	68
2.4.4.2 Github	69
2.4.4.3 Visual Studio Code	69
2.4.4.4 Android Studio	70
2.4.5 APIs REST.....	70
2.4.5.1 FastAPI.....	71
2.4.5.2 Uvicorn.....	71
2.4.5.3 Docker.....	71
2.5 Base de datos	72
2.5.1 Sistema gestor de base de datos	72
2.5.2 Base de datos relacionales.....	72
2.5.2.1 MySQL.....	72
2.5.2.2 MariaDB	73
2.5.3 Base de datos no relacionales.....	73
2.5.3.1 MongoDB.....	73
2.6 Computo en la nube.....	74
2.6.1 Principales modelos de servicio.....	74
2.6.1.1 Infraestructura como servicio (IaaS)	74
2.6.1.2 Plataforma como servicio (PaaS).....	74
2.6.1.3 Software como servicio (SaaS)	75
2.6.2 Amazon Web Services (AWS).....	75
2.6.2.1 Características generales.....	77

2.6.2.2 Amazon EC2 (Elastic Compute Cloud)	78
2.6.2.3 Amazon S3 (Simple Storage Service).....	78
2.6.3 Alternativas a AWS.....	78
2.6.3.1 Google Cloud Platform	78
2.6.3.2 Microsoft Azure.....	79
2.6.3.3 Oracle Cloud	79
CAPÍTULO III. MARCO METODOLÓGICO.....	81
3.1 MOBILE D	81
3.1.1 Fase de exploración	82
3.1.2 Fase de inicialización	82
3.1.3 Fase de producción	83
3.1.5 Fase de pruebas.....	84
3.2 CRISP-DM.....	84
3.2.1 Compresión del problema.....	85
3.2.2 Compresión de datos	85
3.2.3 Preparación de datos	86
3.2.4 Modelado	86
3.2.5 Evaluación del modelo.....	86
3.2.6 Implementación del modelo	86
CAPÍTULO IV. RESULTADOS	88
4.1 Compresión del problema	88
4.2 Compresión de los datos.....	89
4.2.1 Recolección de datos	89
4.2.2 Especies microbianas a considerar	90
4.2.3 Clasificación de imágenes.....	93
4.3 Preparación de datos.....	95
4.3.1 Selección de imágenes.....	95
4.3.2 Normalización y etiquetado	96
4.4 Modelado	98
4.5 Evaluación del modelo	101
4.6 Implementación del modelo	105
CAPÍTULO V. CÓMPUTO EN LA NUBE.....	106
5.1 Creación de la estancia.....	106
CAPÍTULO VI. CONCLUSIONES Y TRABAJO FUTURO	117
REFERENCIAS BIBLIOGRÁFICAS.....	118

ÍNDICE DE TABLAS

Tabla 1	Principales características morfológicas de las colonias microbianas	32
Tabla 2	Parámetros válidos para realizar el conteo	34
Tabla 3	Medios de cultivo sólidos utilizados para el conteo de UFC.	39
Tabla 4	Comparación de modelos de colores más utilizados	47
Tabla 5	Principales parámetros de calidad en una imagen.....	51
Tabla 6	Principales usos de los lenguajes seleccionados.....	61
Tabla 7	Principales aplicaciones y ventajas de librerías utilizadas para el procesamiento de imágenes	65
Tabla 8	Comparación técnica entre múltiples proveedores de nube	76
Tabla 9	Comparativa entre las principales plataformas de cómputo en la nube ...	80
Tabla 10	Conjunto de imágenes empleadas para esta investigación.	89
Tabla 11	Especies microbianas consideradas en la detección automática de colonias.	90
Tabla 12	Clasificación de imágenes.	93
Tabla 13	Hiperparámetros utilizados para el entrenamiento del modelo.	99
Tabla 14	Técnicas de aumentos de datos utilizados para el entrenamiento del modelo.....	100
Tabla 15	Evaluación de modelos de Yolo	101
Tabla 16	Matriz de verdad obtenida de umbrales de IoU en YOLOv11.....	102
Tabla 17	Matriz de verdad obtenida de umbrales de IoU en YOLOv12.....	103
Tabla 18	Matriz de verdad obtenida de umbrales de confianza en YOLOv11. ...	104
Tabla 19	Matriz de verdad obtenida de umbrales de confianza en YOLOv12. ...	104

ÍNDICE DE FIGURAS

Figura 1	Muestra de UFC en una caja Petri	11
Figura 2	Ciclo de formación de colonias.....	33
Figura 3	Principales métodos de conteo de UFC	36
Figura 4	Proceso para llevar a cabo el cultivo en una caja Petri	37
Figura 5	Etapas clave para el procesamiento de imágenes	43
Figura 6	Píxeles que conforman una imagen	44
Figura 7	Ejemplos de resolución espacial	44
Figura 8	Ejemplo de una imagen bidimensional representada en una matriz	46
Figura 9	Ejemplos de resolución radiométrica.....	46
Figura 10	Representación de una imagen con diferente grado de nitidez.....	48
Figura 11	Ejemplo de ruido en una imagen.....	49
Figura 12	Ejemplo de contraste en una imagen	50
Figura 13	Formatos de relación aspecto en una imagen digital	51
Figura 14	Principales algoritmos de aprendizaje automático.....	54
Figura 15	Ciclo de entrenamiento de redes generativas	56
Figura 16	Cálculo de IoU en una caja delimitadora.	57
Figura 17	Valores aproximados del IoU.....	57
Figura 18	Matriz de confusión.	58
Figura 19	Cálculo de la precisión.	59
Figura 20	Cálculo del recall.	60
Figura 21	Cálculo de accuracy.....	60
Figura 22	Flujo de trabajo en Git	69
Figura 23	Diagrama de funcionamiento de una API REST.....	71
Figura 24	Representación de los principales servicios de AWS	75
Figura 25	Red global de regiones de AWS.....	77
Figura 26	Fases de la metodología Mobile-D.	81
Figura 27	Proceso en la fase de exploración.....	82
Figura 28	Proceso en la fase de inicialización.	82
Figura 29	Proceso en la fase de producción.	83
Figura 30	Proceso en la fase de estabilización.	83
Figura 31	Proceso en la fase de pruebas.	84
Figura 32	Diagrama de la metodología CRISP-DM.....	84
Figura 33	Distribución de instancias por clase.	92
Figura 34	Distribución porcentual por clase.....	92
Figura 35	Ejemplo de muestra eliminada.	96
Figura 36	Etiquetado en formato COCO.....	96
Figura 37	Etiquetado en formato YOLO.	97
Figura 38	Visualización de imagen etiquetada.	98
Figura 39	Creación de instancia de EC2	106
Figura 40	Tipo de instancia	107
Figura 41	Creación de llaves de acceso.....	108
Figura 42	Definición de políticas de red.....	109
Figura 43	Asignación de almacenamiento.....	110

Figura 44	Lanzamiento de la instancia.	111
Figura 45	Permitir el tráfico en puertos específicos	111
Figura 46	Conexión a la instancia	112
Figura 47	Actualización e instalación de paquetes	112
Figura 48	Clonar el repositorio de GitHub del proyecto	113
Figura 49	Instalación de los requerimientos de la aplicación.....	113
Figura 50	Creación de la base de datos	114
Figura 51	Ejecución de script para la restauración de la base de datos.....	114
Figura 52	Permitir conexiones remotas a la base de datos	115
Figura 53	Creación de entorno virtual.....	115
Figura 54	Ejecución de servidor de Uvicorn en EC2	116
Figura 55	Prueba de endpoint	116

CAPÍTULO I. GENERALIDADES DEL PROYECTO

En este presente capítulo tiene como objetivo brindar un contexto y justificación del desarrollo del proyecto, este propone una solución tecnológica basada en la visión artificial para automatizar el conteo de unidades formadoras de colonias (UFC) en investigaciones de microbiología. En esta sección se expone los antecedentes, la identificación que origina la investigación, los objetivos propuestos, la justificación del proyecto, sus alcances y limitaciones.

1.1 ANTECEDENTES

El conteo de unidades formadoras de colonias (UFC) es una práctica fundamental en la microbiología, es utilizada para estimar la cantidad de microorganismos viables presentes en una muestra. De acuerdo con Baron et al. (2019), "El recuento de unidades formadoras de colonias (UFC) es considerado un método estándar para evaluar la carga microbiana en muestras de alimentos, aguas y productos farmacéuticos, debido a su simplicidad y reproducibilidad".

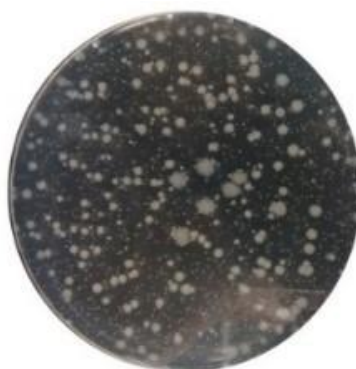
Esta técnica es de gran importancia en múltiples áreas siendo principalmente en la industria alimentaria, la investigación científica y en la producción farmacéutica, donde el control microbiológico es esencial para garantizar la calidad, seguridad y la eficacia de productos y servicios.

En la industria alimentaria, el conteo de unidades formadoras de colonias estima la calidad microbiológica de alimentos y agua, previniendo riesgos de salud asociados con microorganismos patógenos. En el sector de la salud, esta técnica permite el diagnóstico, monitoreo y la evaluación de tratamientos antimicrobianos. Asimismo, en el ámbito de la investigación científica, facilita el estudio de dinámicas microbianas, el desarrollo, la validación de protocolos y pruebas experimentales. (FDA, 2020).

En la Figura 1 muestra una placa Petri con un medio de cultivo, en la que se puede observar el desarrollo de unidades formadoras de colonias, estas se presentan con una gran diversidad de formas, tamaños y distribuidas de manera irregular por toda la superficie del medio.

Figura 1

Muestra de UFC en una caja Petri



Nota. Esta ilustración fue tomada de Redalyc.

Los enfoques tradicionales se basan principalmente en la observación manual de placas Petri. Si bien son métodos ampliamente validados, presentan problemas significativos, como los que se mencionan a continuación:

- Tiempo de procesamiento elevado.
- Subjetividad en la interpretación de resultados.
- Errores humanos derivados de la fatiga o la falta de experiencia.

Para superar estas restricciones se han desarrollado diversas soluciones automáticas que buscan mejorar la exactitud y eficiencia para llevar a cabo este procedimiento. Entre las herramientas comerciales más destacadas se encuentran las siguientes:

- ProtoCOL 3: Es un instrumento para el recuento de colonias, medición de zonas y membranas fabricado por la empresa Synbiosis, donde incluye funciones de análisis, tamaño, color y morfología. Diseñado principalmente

para cumplir con estándares empleada en laboratorios clínicos robustos y de investigación (Synbiosis, 2024).

- BioSpot; Es un instrumento para el recuento de colonias, exploración de datos para auditorías y reportes científicos. Es desarrollado por la empresa BioLogics (BioLogics Inc., s.f.).

Debido a estas limitaciones económicas y de accesibilidad, se han desarrollado herramientas de software libre que proporcionan funciones similares pero limitadas mediante algoritmos de análisis de imagen donde permiten realizar conteos automáticos, destacando las siguientes:

- OpenCFU: Diseñado para realizar la enumeración de objetos circulares como lo son las unidades formadoras de colonias, para ello requiere ajustes manuales para realizar el análisis de acuerdo a la imagen que está siendo procesada y cierto conocimiento técnico para su configuración óptima (Geissmann, 2018).
- ImageJ: Es una plataforma para realizar el análisis de imágenes en el área de investigación desarrollada por el National Institutes of Health (NIH), permite cuantificar mediante parámetros personalizado (Schneider et al., 2012).

Sin embargo, estos dispositivos están diseñados para laboratorios de gran escala y presentan limitaciones importantes, tales como:

1. Altos costos: Restringen su accesibilidad para pequeños laboratorios o instituciones.
2. Requerimientos técnicos: Se requiere conocimientos técnicos avanzados para poder utilizarlos.
3. Falta de portabilidad: La dependencia de equipos especializados dificulta su uso en entornos remotos, o lugares no controlados.
4. Limita la integración con otros dispositivos: Restringe la aplicación en tareas de campo con infraestructuras específicas.

De acuerdo Zhang et al. (2020), "Los sistemas de visión por computadora pueden superar las limitaciones manuales, proporcionando mayor precisión, rapidez y eliminando la subjetividad humana", en el campo de la microbiología, la visión artificial se perfila como una herramienta que no solo facilita el conteo de UFC, sino que también mejora la reproducibilidad de los resultados, una necesidad crucial en el desarrollo de una investigación y en la producción industrial.

Por ello, la visión artificial ha demostrado ser una herramienta crucial en múltiples aplicaciones, entre las que se destacan:

- Monitoreo ambiental: "Los sistemas de visión por computadora permiten la detección automatizada de contaminantes microbianos en superficies y aire, superando las limitaciones de los métodos tradicionales" (Liu et al., 2021).
- Diagnósticos clínicos: De acuerdo con Esteva et al. (2017), el análisis de imágenes mediante deep learning ha revolucionado la identificación de patrones microbianos en muestras clínicas, reduciendo tiempos de diagnóstico y aumentando la precisión.
- Controles de calidad industrial: "En la industria alimentaria y farmacéutica, la visión artificial estandariza el conteo de colonias microbianas, garantizando cumplimiento normativo y reproducibilidad" (Bruno et al., 2019).
- Vehículos autónomos: De acuerdo con Bojarski (2018), "La integración de algoritmos de visión artificial en sensores LiDAR y cámaras es fundamental para la navegación segura de vehículos autónomos en entornos dinámicos"
- Automatización de procesos: "Robots equipados con visión artificial optimizan tareas repetitivas en manufactura, desde inspección de piezas hasta clasificación de materiales" (Krizhevsky et al., 2018).
- Gestión de recursos: "Drones con visión artificial monitorizan cultivos y recursos hídricos, proporcionando datos en tiempo real para una agricultura sostenible" (Kamilaris & Prenafeta, 2018).

Dado a lo anterior, el avance en algoritmos de procesamiento de imágenes y visión artificial ha revolucionado de manera específica, algunos enfoques aplicables a esta investigación:

- Segmentación tradicional: el método Watershed y Otsu han demostrado efectividad en la separación de colonias, aunque presentan dificultades ante ruido, superposición de colonias y variaciones de iluminación.
- Modelos basados en inteligencia artificial donde las redes neuronales convolucionales (CNN) permiten clasificar y contar colonias.

Algunos de los estudios recientes que han demostrado el potencial de la visión artificial en microbiología, destacando los siguientes:

- Zhang et al., (2020), "Deep learning for imaging and detection of microorganisms". Este artículo revisa las técnicas de Deep learning como lo son las redes neuronales convolucionales para el análisis de imágenes de microbianas, la detección automática de patógenos y clasificación de microorganismos.
- Christiansen et al. (2021), "Deep learning enables rapid identification of bacteria in microscopy images". Este estudio propone un método basado en inteligencia artificial donde las bacterias podrán ser identificadas sin la necesidad de tinciones químicas, mediante la implementación de redes neuronales convolucionales
- Litjens et al. (2017), "A survey on deep learning in medical image analysis" Este estudio discute la aplicación de Deep learning para llevar a cabo el análisis de imágenes médicas, como lo es la detección de microorganismos en histopatología.
- Ho et al. (2020), "Raman spectroscopy and machine learning for bacterial identification". Este estudio explora las técnicas de espectroscopía Raman y aprendizaje automático, para poder detectar patógenos bacterianos sin la necesidad de cultivos mediante la implementación de modelos como SVM o CNN.

Dado a lo anterior, este proyecto propone el desarrollo de una solución innovadora que combine visión artificial y accesibilidad móvil para el conteo automático de unidades formadoras de colonias cuyas principales aportaciones incluyan:

1. Aplicación multiplataforma (Android e iOS) para garantizar su accesibilidad en diferentes entornos.
2. Procesamiento en tiempo real mediante algoritmos optimizados, permitiendo realizar conteos directamente desde una fotografía tomada desde la cámara de un dispositivo móvil.
3. Registro estadístico automatizado, útil para análisis y generación de reportes en tiempo real.
4. Conformidad con la norma NOM-092-SSA1-1994.

Tal y como se menciona anteriormente, este proyecto se desarrollará en conformidad con la norma mexicana NOM-092-SSA1-1994, la cual establece los métodos para la determinación de la cuenta de bacterias aerobias en placa en productos de consumo humano. Esta norma es eficiente para garantizar la seguridad y la calidad microbiológica en la industria alimentaria.

El software propuesto integrará algoritmos de visión artificial que cumplen con los requisitos específicos de esta norma asegurando que los conteos automáticos sean precisos. Al utilizar técnicas avanzadas de procesamiento de imágenes, el software podrá:

1. Detectar y contar colonias de manera precisa y eficiente: Superando las limitaciones de la observación manual y reduciendo el margen de error.
2. Automatizar el registro y la documentación de resultados: Facilitando la generación de reportes que cumplan con los estándares requeridos por la norma, asegurando la trazabilidad y la transparencia en los procesos.
3. Ofrecer herramientas de análisis en tiempo real: Permitirá a los usuarios monitorear y evaluar los resultados de manera rápida, mejorando la capacidad de respuesta ante posibles contaminaciones o desviaciones de calidad.

4. Garantizar la reproducibilidad y consistencia de los resultados: Minimizar la variabilidad derivada de la interpretación subjetiva, proporcionando datos más fiables para la toma de decisiones.

El cumplimiento de la NOM-092-SSA1-1994 también implica que el software se desarrollará considerando las prácticas y procedimientos recomendados. Al adherirse a esta normativa, el proyecto no solo mejorará la eficiencia y precisión en el conteo de unidades formadoras de colonias, sino que también contribuirá a elevar los estándares de seguridad y calidad en la industria alimentaria y otros sectores relevantes.

1.2 IDENTIFICACIÓN DEL PROBLEMA

El conteo de unidades formadoras de colonias UFC es una actividad esencial aplicada en el área de microbiología, empleada para evaluar, determinar la presencia y cantidad de microorganismos en múltiples aplicaciones (Bruno et al., 2019). En la actualidad esta es una técnica estándar que es llevada a cabo mediante un conteo manual, siendo un proceso que se realiza de manera lenta, propensa a errores humanos, además que puede retrasar la toma de decisiones, siendo un ejemplo la liberación de productos o la identificación de patógenos.

Esta problemática afecta principalmente a laboratorios científicos, la industria alimentaria, farmacéutica y el sector médico. Estos sectores dependen de resultados rápidos y precisos con el fin de garantizar la calidad y seguridad de sus procesos y productos.

El impacto de esta problemática es considerable en dos áreas clave de la microbiología: los exámenes microbiológicos y los exámenes de áreas estériles. En los exámenes microbiológicos, el conteo de UFC se utiliza para controlar contaminaciones en alimentos, agua, productos farmacéuticos y muestras clínicas. Por otro lado, en los exámenes de áreas estériles busca determinar la ausencia de microorganismos es fundamental para mantener estándares (Pinto et al., 2021). Para llevar a cabo la propuesta de resolución de esta problemática se considera que es viable debido a los avances en visión artificial, procesamiento de imágenes y la capacidad de dispositivos móviles. La implementación de estas tecnologías permite diseñar y desarrollar un software e integrarlo mediante una aplicación móvil personalizable, accesible y escalable para la automatización del conteo de UFC, reduciendo los tiempos de análisis y mejorando la precisión.

La factibilidad de implementación es alta, ya que los algoritmos de procesamiento de imágenes y aprendizaje automática, ofreciendo una solución económica. Esta solución no solo incrementaría la precisión del proceso, sino que también

optimizaría el tiempo y los recursos humanos, reduciendo los errores causados por la intervención manual.

1.3 OBJETIVOS

1.3.1 Objetivo general

Desarrollar un software basado en visión artificial que permita el conteo de unidades formadoras de colonias (UFC) de microorganismos para eficientizar la toma de decisiones en estudios científicos.

1.3.2 Objetivos específicos

1. Investigar el estado del arte sobre métodos de conteo de UFC.
2. Diseñar un algoritmo de procesamiento de imágenes para el conteo automático de UFC.
3. Implementar una aplicación móvil que permita capturar imágenes para realizar el conteo automático de UFC.
4. Validar el sistema mediante pruebas comparativas entre los resultados obtenidos automáticamente y los conteos manuales realizados.
5. Analizar los resultados obtenidos para evaluar su aplicabilidad en múltiples entornos.

1.4 JUSTIFICACIÓN

El control microbiológico en áreas estériles es de gran importancia en industrias donde la contaminación puede representar un riesgo ya sea para la salud, la integridad de productos y la seguridad, siendo principalmente en la industria farmacéutica, hospitales, laboratorios clínicos y laboratorios de investigación (Sutton, 2020).

Sin embargo, el monitoreo y conteo tradicional de unidades formadoras de colonias (UFC) en estas áreas es un proceso que requiere mucho tiempo y suele depender del trabajo manual en laboratorios, lo cual puede retrasar la detección de posibles contaminaciones. Esta aplicación móvil responde a la necesidad de contar con una herramienta rápida, portátil y precisa para el conteo de UFC en tiempo real, complementando los métodos convencionales y facilitando un monitoreo más constante y ágil de la calidad microbiológica en áreas críticas.

La finalidad de este proyecto es automatizar y optimizar el proceso para el conteo microbiológico de unidades formadoras de colonias en áreas estériles, para ello reduciendo la dependencia del análisis manual y permitiendo obtener resultados de manera rápida. Con esta aplicación, el personal de control de calidad podrá realizar conteos directamente en el área de trabajo. Esto permitirá una evaluación constante de las condiciones de limpieza y esterilidad, ayudando a determinar la presencia de microorganismos en entornos donde la contaminación podría comprometer la seguridad y calidad de los productos, así como la salud de los usuarios.

Las problemáticas que resuelve:

- Tiempos de respuesta prolongados: Actualmente, el proceso manual de conteo y análisis de muestras en laboratorio, para ello se propone mejorar los tiempos de respuesta y la toma de decisiones.
- Dependencia del análisis manual: El conteo manual está sujeto a errores humanos y es un proceso laborioso, especialmente en ambientes donde se

debe realizar monitoreo frecuente. La visión artificial reducirá los errores asociados al conteo manual, mejorando la precisión de los resultados.

- Cumplimiento normativo y trazabilidad: Las normativas de calidad y seguridad en áreas estériles requieren un monitoreo constante de los niveles microbiológicos. Para ello se permitirá registrar, almacenar los datos, asegurando la trazabilidad y para el cumplimiento de normas como la NOM-092-SSA1-1994
- Aplicación de tecnología e innovación al área de microbiología.

1.5 ALCANCES Y LIMITACIONES

1.5.1 Alcances

La investigación se centrará en el diseño y desarrollo de un software basado en visión artificial para el conteo automático de unidades formadoras de colonias (UFC) en contextos microbiológicos, utilizando imágenes de placas de Petri u otros soportes microbiológicos comunes.

Se llevará a cabo la implementación de técnicas de procesamiento digital de imágenes, algoritmos de segmentación para identificar y cuantificar colonias microbiológicas. Esta solución estará implementada en una aplicación móvil con el fin de garantizar la portabilidad y facilidad de uso en laboratorios con recursos limitados.

Además de garantizar el almacenamiento de todos los conteos de los exámenes realizados, permitiendo generar un informe estadístico e informativo que permita analizar tendencias, resultados acumulados y otros datos relevantes para la toma de decisiones.

Esta aplicación móvil será desarrollada para los principales sistemas operativos de los dispositivos inteligentes siendo Android e iOS, asegurando una mayor compatibilidad para los dispositivos móviles de la actualidad. Esta aplicación móvil propondrá una interfaz gráfica intuitiva para que usuarios no especializados puedan operar dicha aplicación.

Para llevar a cabo la validación del software será probado con imágenes reales de cultivos microbiológicos para evaluar su desempeño en escenarios prácticos dentro de un laboratorio.

1.5.2 Limitaciones

El proyecto se desarrollará dentro del tiempo establecido para la tesis, este periodo consta desde agosto del 2024 hasta agosto del 2026. El conteo de unidades formadoras de colonias estará limitado a todas aquellas que sean visibles además no se contemplará la identificación específica de microorganismos ni su clasificación. Adicional no se abordarán los casos de cultivos complejos o colonias muy fusionadas que requieran técnicas de separación.

Las pruebas se realizarán con muestras disponibles en un laboratorio, lo que puede limitar la generalización de los resultados a otros entornos microbiológicos con condiciones distintas.

Se utilizarán equipos móviles de uso común, por lo que el rendimiento del software dependerá de las capacidades de procesamiento de estos dispositivos. La calidad de las imágenes capturadas dependerá de las condiciones de iluminación y la cámara del dispositivo móvil, lo que podría afectar la precisión del conteo.

El acceso a bases de datos extensas de imágenes microbiológicas podría estar limitado, lo que restringiría el entrenamiento y validación del sistema.

1.6 PREGUNTA DE INVESTIGACIÓN

¿De qué manera el desarrollo e implementación de un software basado en visión artificial impacta la precisión, eficiencia y consistencia del conteo de unidades formadoras de colonias (UFC) de microorganismos, contribuyendo a la toma de decisiones en estudios científicos?

1.6.1 Preguntas específicas

- 1.- ¿Cómo se puede diseñar un algoritmo eficiente de procesamiento de imágenes para identificar y contar automáticamente las unidades formadoras de colonias (UFC) de microorganismos en diferentes tipos de muestras microbiológicas?
- 2.- ¿Cuál es el impacto del uso de visión artificial en la precisión del conteo de UFC en comparación con los métodos tradicionales manuales en diferentes condiciones de cultivo microbiológico?
- 3.- ¿Cómo influye el tipo de cámara y la calidad de la imagen en la precisión y eficiencia del conteo automático de UFC a través de una aplicación móvil?
- 4.- ¿Cuáles son los desafíos técnicos y limitaciones en el desarrollo de un software móvil multiplataforma (Android e iOS) para el conteo automático de UFC mediante visión artificial?

1.7 HIPÓTESIS

El uso de un software basado en visión artificial para el conteo automatizado de unidades formadoras de colonias (UFC) de microorganismos permitirá obtener resultados más precisos, rápidos y consistentes en comparación con los métodos manuales, lo que facilitará una toma de decisiones más eficiente en estudios científicos.

Variables dependientes

1. Precisión de conteo: Grado de exactitud del conteo automático respecto al real.
 - a. Unidad de medida: Porcentaje.
 - b. Métrica: Error Medio Porcentual (EM%).
2. Eficiencia del conteo: Rapidez con la que el sistema genera un resultado.
 - a. Unidad de medida: Segundos(s).
 - b. Métrica: Tiempo Promedio de Procesamiento (TPP).
3. Consistencia del conteo: Variabilidad del sistema al analizar imágenes similares o iguales.
 - a. Unidad de medida: Porcentaje.
 - b. Métrica: Desviación estándar del EM%.

Variables independientes

1. Calidad de imagen: Nitidez, enfoque, contraste, ruido.
2. Resolución de imagen: Tamaño en píxeles de la imagen.
3. Condiciones de iluminación: intensidad de luz al capturar la imagen.
4. Cantidad de UFC en la imagen: Densidad de colonias presentes en la muestra.
5. Dispositivo de captura: Características físicas de la cámara del dispositivo.

1.8 RIESGOS Y CONSIDERACIONES ÉTICAS

1.8.1 Riesgos

Al igual que en cualquier proyecto de investigación, es necesario considerar todos los potenciales riesgos que puedan presentarse durante su desarrollo, con el objetivo de prevenir dificultades que perjudiquen la finalización del proyecto o provoquen un retraso. Por lo tanto, se han detectado los siguientes posibles riesgos que se presentarán durante el desarrollo del proyecto, junto con la propuesta de estrategias que posibiliten enfrentarlos de forma proactiva y asegurando la ejecución exitosa y sostenible del mismo:

1. Riesgos técnicos

- a. Insuficiente calidad de imagen: Cuando una imagen presenta desenfoque, iluminación deficiente o no satisface los estándares básicos para su funcionalidad.
- b. Negativos o falsos positivos en el algoritmo: Estimación imprecisa debido a fallos del modelo de visión artificial.
- c. Errores en el control o pérdida de información: Dificultades en la transferencia o almacenamiento de imágenes y resultados.
- d. Errores en el servidor: Alteraciones en el servicio que dificulten la utilización del sistema.

2. Riesgos operativos

- a. Falta de personal humano: Ausencia de respaldo técnico para concluir el proceso de desarrollo.
- b. Dificultades debido a la falta de coordinación: Problemas en la coordinación del equipo o del calendario.

Estrategias de mitigación:

1. Riesgos técnicos:

- a. Indicaciones precisas para la captura de una imagen.
- b. Aplicación de técnicas de procesamiento a una imagen.
- c. Validación que la imagen cuente con los requisitos necesarios

- d. Comprobación mediante el conteo manual con muestras diversas.
 - e. Ajuste de límites y segmentación en la imagen.
 - f. Verificación del servicio y copias de seguridad regulares.
2. Riesgos operativos
- a. Planificación realista con márgenes de retraso.
 - b. Reuniones periódicas de seguimiento del proyecto.

1.8.2 Consideraciones éticas

Entre las consideraciones éticas que este proyecto debe abordar, se han identificado las siguientes:

1. Privacidad y confidencialidad de información personal:
 - a. Solo se almacenan los datos indispensables (principio de minimización).
 - b. Los datos de usuario se encriptan y se restringen para propósitos de rastreabilidad académica.
 - c. Todo tránsito de datos se realiza sobre HTTPS.
2. Consentimiento consciente e informado:
 - a. Cada muestra junto con su imagen vinculada necesita tener la aprobación del usuario que registra la muestra.
3. Acceso equitativo y beneficio social:
 - a. Promoción gratuita para laboratorios educativos y personas interesadas.
 - b. Documentación transparente para disminuir obstáculos de adopción.
4. Uso responsable:
 - a. Incluye un aviso que EasyCountCol no es un dispositivo médico ni emite diagnósticos.

CAPÍTULO II. MARCO TEÓRICO

En el presente capítulo se hará una recopilación de las consideraciones que sustentan este proyecto de investigación con el fin de sentar las bases que guíen el planteamiento y propuesta de solución.

Esta investigación parte desde el análisis de los principios microbiológicos para comprender la importancia de las unidades formadoras de colonias (UFC) donde incluye la clasificación, condiciones de crecimiento, medios de cultivo, técnicas de siembra y dilución. Adicional se exploran los fundamentos de procesamiento digital de imágenes, visión artificial como herramientas clave para la detección y conteo de colonias bacterianas.

También se contempla el desarrollo de una aplicación móvil multiplataforma donde permita al usuario capturar o cargar imágenes desde un dispositivo móvil. Para ello, se emplea Flutter con el fin de garantizar la compatibilidad con sistemas Android e iOS, y facilitando la comunicación con el backend mediante endpoint.

Este conjunto de componentes teóricos y tecnológicos proporciona la base para el desarrollo de una solución que automatiza el conteo de microorganismos y favorezca la optimización de los procesos de análisis en contextos científicos.

2.1 Fundamentos microbiológicos

Este apartado proporciona las bases para comprender la importancia de los microorganismos en los procesos de análisis microbiológico, principalmente en aquellos conceptos relevantes para realizar el conteo de unidades formadoras de colonias (UFC). Dentro los cuales son: la clasificación de los microorganismos, sus condiciones de crecimiento, la importancia de los medios de cultivo, la relación entre las colonias observables y la cantidad de microorganismos viables en un muestreo.

2.1.1 Introducción a la microbiología

"La microbiología es disciplina científica que estudia los microorganismos, entidades biológicas demasiado pequeñas para ser observadas sin aumento óptico o electrónico, incluyendo bacterias, arqueas, virus, hongos microscópicos, protozoos y algas unicelulares. Su estudio abarca no solo su morfología y fisiología, sino también su genética, ecología, evolución y su impacto en la salud humana, la industria y el medio ambiente" (Madigan et al., 2021).

Según Stanier et al. (2024), "Si un objeto que tiene un diámetro inferior a 1 mm, el ojo humano no percibirlo, por lo tanto, aquellos organismos que tienen un diámetro de 1mm o inferior son considerados microorganismos y caen dentro del amplio dominio de la microbiología".

"El estudio de los microorganismos ha brindado en los últimos años grandes aportaciones para la solución de problemas biológicos básicos. Su fácil manejo, el crecimiento rápido, la gran capacidad adaptativa y otras características han hecho de los microorganismos el objeto de estudio preferido de la Bioquímica y la Genética" (Schlegel, 1997).

2.1.2 Aplicaciones de la microbiología en sectores clave

La microbiología tiene una relevancia fundamental en múltiples sectores, ya que los microorganismos influyen en la salud, la industria, el ambiente y la producción de bienes y servicios.

2.1.2.1 Sector clínico

"La microbiología clínica es una ciencia de juicio interpretativo que responde a las necesidades clínicas del médico tratante, con el fin de identificar el agente etiológico de una infección y establecer la actividad in vitro de las drogas antimicrobianas contra los microorganismos identificados" (Buchan et al., 2022). El laboratorio de microbiología clínica proporciona información para el diagnóstico, tratamiento y control de enfermedades infecciosas, contribuyendo en la toma de decisiones médicas y a la salud pública.

2.1.2.2 Industria alimentaria

En el sector de la alimentación, la microbiología tiene un rol importante tanto en la elaboración como en la preservación y seguridad de los alimentos. En los procesos de fermentación, los microorganismos beneficiosos, como las bacterias ácido-lácticas y las levaduras, resultan importantes para la producción de productos como yogur, queso, pan, cerveza y vino, dado que modifican los sustratos y potencian las propiedades organolépticas. No obstante, también hay microorganismos dañinos y modificadores que pueden poner en riesgo la calidad y seguridad de los alimentos, lo que requiere la implementación de técnicas de control como la pasteurización, el salado o la aplicación de bacteriocinas (Madigan et al., 2021).

2.1.2.3 Industria farmacéutica

En el sector farmacéutico desempeña un papel primordial en el desarrollo y control de calidad en la industria. A través de la bioquímica microbiana, es posible la producción de medicamentos siendo un ejemplo los antibióticos, para ello los microorganismos son alterados genéticamente para producir sustancias.

En el control de calidad se utiliza para garantizar que los productos farmacéuticos estén libres de contaminación por microorganismos. Debido a que los medicamentos son susceptibles a la contaminación microbiana ya que son un ambiente nutritivo para el crecimiento de diferentes microorganismos. Para ello se realizan pruebas microbiológicas durante el proceso de producción para garantizar la seguridad y eficacia de los fármacos (Calderón et al., 2022).

2.1.2.4 Medio ambiente

En el sector del medio ambiente, los microorganismos son actores clave en la preservación de los ecosistemas, debido a su participando en ciclos biogeoquímicos, biorremediación y regulación climática (Madigan et al., 2021).

Para ello hacen uso de nutrientes vitales como el nitrógeno, carbono y azufre a través de procedimientos como la fijación biológica y la degradación de materia orgánica, además de realizar la limpieza en entornos contaminados a través de la biorremediación (EPA, 2020). Su impacto en el clima es importante debido a que

permiten absorber CO₂ y liberar gases esenciales para el efecto invernadero (IPCC, 2021).

Según Grossart et al. (2019), la presencia de esporas microbianas en terrenos, aguas y aire fortalece su resistencia ambiental y las convierte en una herramienta alentadora frente a la polución y el calentamiento global.

2.1.3 Microorganismos

De acuerdo con Prescott et al. (2020), "Son formas de vida diversas, generalmente unicelulares, que habitan en todos los ambientes conocidos", además de que solo puede observarse desde un microscopio. Estos organismos pueden ser unicelulares o pluricelulares, algunos de estos pueden ser beneficiosos (probióticos) mientras que el resto son patógenos (causante de enfermedades).

2.1.3.1 Clasificación de los microorganismos

Los microorganismos se clasifican en cuatro grandes grupos según sus características estructurales, genéticas y funcionales: bacterias, virus, hongos y parásitos (Madigan et al., 2021). A continuación, se detallan sus propiedades:

- Bacterias: "Son organismos unicelulares que pertenecen al grupo de los procariontes; esto quiere decir que carecen de un núcleo celular y de orgánulos. Las bacterias miden entre 0.5 y 5 μ de longitud; son tan pequeñas que es imposible verlas a simple vista" (Contreras et al., 2017).
- Virus: "Son organismos más pequeños, no son considerados células debido a que carecen de muchos de los atributos de éstas, no cuentan con un metabolismo propio. La estructura de los virus es muy diversa y varía en tamaño, forma y composición química" (Preciado, 2020).
- Hongos: "Son organismos eucariotas que pueden ser unicelulares (levaduras) o pluricelulares (mohos) siendo además heterótrofos. Su reproducción es por gemación, esporulación o fragmentación en el medio extracelular" (Geovana et al., 2017).

- **Parásitos:** Son organismos eucariotas que crecen y se alimentan a expensas de otro organismo diferente, este obtiene nutrientes y protección física sin hacer ninguna contribución a la supervivencia de su anfitrión” (Mier et al. ,2019).

2.1.3.2 Morfología de las colonias microbianas

Las colonias microbianas presentan características macroscópicas distintivas que permiten su identificación preliminar en el laboratorio, estas características se evalúan después del crecimiento en medios de cultivo y son esenciales para diferenciar grupos microbianos antes de las pruebas. En la Tabla 1 describe las principales características morfológicas.

Tabla 1

Principales características morfológicas de las colonias microbianas

Características	Descripción	Ejemplos
Forma	Circular, irregular, filamentosa o rizoide.	E. coli (circular), Bacillus (irregular)
Elevación	Plana, convexa, umbonada o crateriforme.	Staphylococcus (convexa)
Borde	Liso, ondulado, lobulado o filamentoso.	Pseudomonas (ondulado)
Superficie	Lisa, rugosa, brillante o mate.	Mycobacterium (rugosa)
Color	Pigmentación específica.	Serratia marcescens (rojo)
Opacidad	Translúcida, opaca o iridiscente.	Micrococcus luteus (amarillo opaco)

Nota. Esta tabla muestra las características morfológicas, una descripción y ejemplos de las anteriormente mencionadas.

Estas propiedades están determinadas genéticamente y pueden verse influenciadas por factores externos como el tipo de medio, la temperatura y la cantidad de nutrientes disponibles. Aunque no son determinantes por sí solas, permiten una orientación diagnóstica inicial antes de aplicar métodos moleculares o bioquímicos más específicos (Madigan et al., 2021).

2.1.4 Unidades Formadoras de Colonias (UFC)

Las Unidades Formadoras de Colonias (UFC) representan una unidad de medida utilizada en microbiología para estimar el número de microorganismos viables en una muestra. Una UFC se define como: "Una o más células microbianas que dan origen a una colonia visible en un medio de cultivo sólido bajo condiciones controladas de incubación" (Madigan et al., 2021).

2.1.4.1 Principios básicos

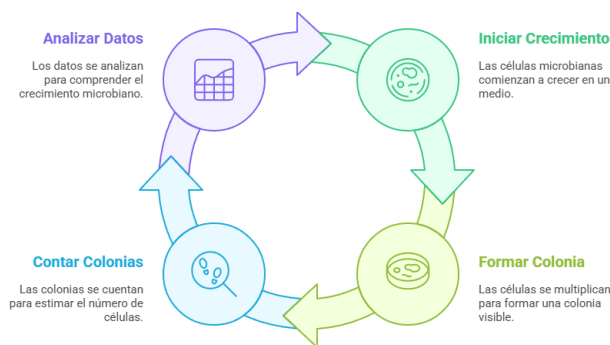
Los principios básicos para realizar el conteo de UFC son los siguientes:

- Viabilidad: Solo se contabilizan aquellas células vivas con la habilidad para proliferar y formar colonias, sin embargo, este método no reconoce las células muertas.
- Dilución seriada: Para la obtención de resultados, se realizan diluciones sucesivas de la muestra con el objetivo de alcanzar un rango adecuado de colonias contables por placa, generalmente entre 30 y 300, evitando así la saturación o ausencia de colonias.
- Especificidad: El progreso e identificación es fundamental en el medio de cultivo y condiciones de incubación.

En la Figura 2 se presenta un ciclo del crecimiento y análisis microbiológico representado mediante un diagrama circular con cuatro etapas interconectadas.

Figura 2

Ciclo de formación de colonias



Nota. Esta ilustración fue elaborada por Napkin.

2.1.4.2 Parámetros para un conteo valido

Para que el conteo de unidades formadoras de colonias sea válido y representativo, se deben cumplir ciertos parámetros, uno de ellos es seleccionar placas que contengan un número adecuado de colonias para minimizar errores de conteo y evitar la superposición de colonias, además se recomienda realizar diluciones seriadas de la muestra para asegurar que al menos una de las placas sembradas este dentro del rango óptimo de conteo, por último el volumen sembrado debe ser conocido y constante, normalmente 0,1 ml o 1 ml por placa (SAG, 2022).

En la Tabla 2 describe los principales parámetros con su rango óptimo para poder realizar cabo el conteo de unidades formadoras de colonias.

Tabla 2

Parámetros válidos para realizar el conteo

Parámetro	Requisito
Rango	30–300 colonias/placa.
Diluciones adecuadas	Dilución seriada en factores de 10.
Medios de cultivo	Selectivos/no selectivos según objetivo.
Condiciones de incubación	Temperatura y tiempo estandarizado.
Volumen de siembra	0.1mL o 1mL

Nota. Conteos fuera del rango se reportan como "demasiado numerosas para contar" o "menos de 30".

2.1.4.3 Expresión de resultados

Los resultados del conteo de unidades formadoras de colonias deben reportarse de acuerdo con el tipo de muestra analizada, siguiendo:

- Muestras líquidas: Se expresa UFC por mililitro (UFC/ml).
- Muestras solidas o semisólidas: Se expresa UFC por gramo (UFC/g).

Como se demuestra en la ecuación 1, el cálculo de unidades formadoras de colonias (UFC) por mililitro (mL) o gramo (g) se realiza mediante la siguiente expresión:

$$\frac{UFC}{mL} \left(o \frac{UFC}{g} \right) = \frac{N\acute{u}mero\ de\ colonias}{Volumen\ inoculado(mL) \times Factor\ de\ diluci\acute{o}n} \quad (1)$$

Esta fórmula permite estimar la concentración de microorganismos viables en una muestra, considerando los siguientes

- Número de colonias: corresponde a la cantidad de colonias visibles que crecieron en la placa de Petri tras la incubación.
- Volumen inoculado (mL): es el volumen de muestra que fue sembrado sobre el medio de cultivo.
- Factor de dilución: es el valor que representa cuánto se diluyó la muestra original.

Como se muestra en la ecuación 2, se aplica la fórmula de la ecuación 1 con valores específicos:

$$\frac{150}{0.1 \times 10^{-5}} = 1.5 \times 10^7 \text{ UFC/mL} \quad (2)$$

Reglas de redondeo:

- Los resultados numéricos deben ser expresados en notación científica, ejemplo: 2.4×10^3 .
- Conteos bajos, si el conteo es < 30 , reportar como $< 30 \text{ UFC/mL}$.
- Redondeo numérico: si el dígito al último decimal es ≤ 5 se redondea hacia arriba, ejemplo: $248 = 2.5 \times 10^2 \text{ UFC/mL}$ en caso contrario $144 = 1.4 \times 10^2 \text{ UFC/mL}$.

Resultados no cuantificables:

- Sin crecimiento: Se indica como "0 UFC/mL" o "Ausencia en (volumen/masa analizada).
- Sobrecrecimiento Se indica como $>$ rango máximo de conteo UFC/mL .

Los límites microbiológicos varían según el tipo de muestra, a continuación, se describen las categorías más comunes:

- Alimentos: Los límites son establecidos según tipo de producto (ejemplos: leche $< 10,000 \text{ UFC/mL}$ y carne fresca $< 100,000 \text{ UFC/g}$ (ICMSF, 2022).
- Agua: El límite es $< 100 \text{ UFC/mL}$ para consumo humano (OMS, 2017) o 0 UFC/100mL en coliformes fecales.
- Productos farmacéuticos: El límite es $< 10 \text{ UFC/g}$ para ingredientes no estériles.

2.1.5 Métodos tradicionales para el conteo microbiano

Los métodos tradicionales de conteo microbiano son técnicas utilizadas en microbiología para determinar la cantidad de microorganismos presentes en una muestra. Estos métodos son ampliamente empleados en laboratorios de control de calidad, investigación y diagnóstico (Madigan et al., 2021).

En la Figura 3 muestra los principales métodos de conteo de Unidades Formadoras de Colonias (UFC) utilizados en microbiología, clasificados según su especificidad y complejidad.

Figura 3

Principales métodos de conteo de UFC



Nota. Esta ilustración fue elaborada por Napkln.

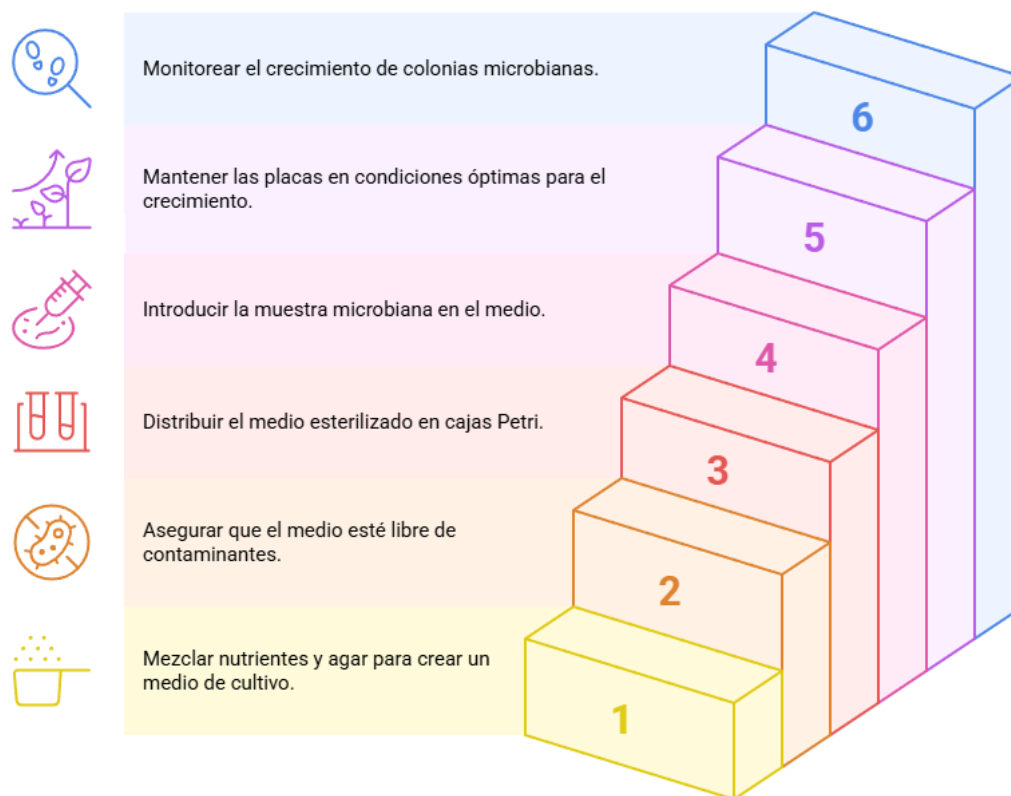
2.1.5.1 Técnica de cultivo en caja Petri

La manera más común de cultivar las unidades formadoras de colonias es mediante el uso de placa Petri. Sin embargo, esta técnica implica sembrar una muestra disuelta en un medio de cultivo sólido, normalmente en agar nutritivo e incubarla bajo condiciones de temperatura y tiempo. Este procedimiento es visto como un estándar en microbiología por su habilidad para identificar microorganismos viables concretos, distinguirlos morfológicamente y llevar a cabo pruebas de sensibilidad a antimicrobianos (Madigan et al., 2021).

En la Figura 4 muestra de manera ilustrativa los pasos para preparar y manejar un cultivo microbiológico en laboratorio, organizados como una escalera de seis niveles.

Figura 4

Proceso para llevar a cabo el cultivo en una caja Petri



Nota. Esta ilustración fue elaborada por Napkln.

2.1.5.2 Medios de cultivos más utilizados

Los medios de cultivo son combinaciones nutritivas empleadas en el desarrollo de microorganismos.

Se pueden clasificar según su función objetivo:

- Medios selectivos: Son aquellos que contienen componentes que permiten el crecimiento de una especie específica, ya que inhiben el crecimiento de otras especies, se utilizan principalmente para aislar y cultivar microorganismos específicos (Solmeglas, 2023).
- Medios diferenciales: Son aquellos que permiten distinguir entre diferentes especies, se utilizan principalmente para identificar especies de microorganismos y para detectar cambios en su comportamiento, contienen indicadores como colorantes que permiten la diferenciación como resultado de una reacción química (COFEPRIS, 2023).
- Medios enriquecidos: De acuerdo con Rodríguez (2018), “Son aquellos que estimulan el crecimiento de los microorganismos y permiten el desarrollo de la población microbiana a partir de un número reducido de células de determinados gérmenes que pueden encontrarse en presencia de otros que, hallándose en condiciones favorables, pudieran frenar el desarrollo de los primeros”.

Se pueden clasificar según consistencia física:

- Sólido: Son aquellos en los que los nutrientes están en forma sólida.
- Líquido: Son aquellos en los que los nutrientes están en forma líquida.

Se pueden clasificar según origen:

- Natural: Son aquellos que son preparados a partir de sustancias naturales como extractos de tejidos, levadura.
- Sintéticos: Son aquellos que tienen una composición química definida cualitativa y cuantitativamente.

En la Tabla 3 muestra los tipos de agar más frecuentemente utilizados para el conteo de Unidades Formadoras de Colonias (UFC) en aplicaciones como el control microbiológico de superficies, alimentos y agua.

Tabla 3

Medios de cultivo sólidos utilizados para el conteo de UFC.

Tipo de agar	Uso principal	Aplicación	Representación
Agar Plate Count (PCA)	Recuento de bacterias viables totales.	Alimentos, agua, aire, superficies estériles.	
Agar Nutriente	Conteo general de bacterias no exigentes.	Control de calidad general.	
Agar MacConkey	Aislamiento y recuento de enterobacterias.	Aguas residuales, alimentos.	
Agar Manitol Salado (MSA)	Detección de Staphylococcus aureus.	Ambientes estériles, superficies, hospitales.	
Agar Sabouraud Dextrosa	Recuento de hongos y levaduras.	Aire, cosméticos, ambientes húmedos.	
Agar VRBA	Recuento de coliformes totales.	Alimentos, productos lácteos.	

"La selección del medio de cultivo depende de las características metabólicas del microorganismo de interés, así como de los objetivos del estudio (detección, cuantificación o identificación)" (Cabrales et al., 2020).

2.1.5.3 Limitaciones del conteo manual

A pesar de su utilidad, los métodos tradicionales presentan desventajas:

- Subjetividad del operador: El conteo de colonias puede variar entre operadores que realizan el conteo, normalmente cuando las colonias son pequeñas o están sobrepuestas.
- Tiempo de realización: El proceso completo (siembra, incubación, conteo) cuenta con un tiempo entre 24 a 72 horas.
- Condiciones de cultivo: Algunos microorganismos son difíciles de cultivar.
- Error humano: Comienza desde la preparación de la dilución hasta el conteo de colonias.

2.2 Procesamiento digital de imágenes

El procesamiento digital de imágenes (PDI) es el conjunto de técnicas y procesos que permiten resaltar información que puede proporcionar una imagen mediante el uso de una computadora. Esta área especializada de la computación tiene dos principales aplicaciones: mejorar la calidad visual de la imagen para facilitar su interpretación por parte de los humanos, y procesar automáticamente los datos presentes en un entorno mediante sistemas de percepción autónoma (Burger et al., 2021).

2.2.1 Aplicaciones del procesamiento digital de imágenes

El procesamiento digital de imágenes (PDI) tiene numerosas aplicaciones en diversos campos, muchas de ellas respaldadas por investigaciones y publicaciones científicas. A continuación, se presentan algunas de las principales aplicaciones:

- Medicina: Técnicas como el filtrado, la segmentación y la reconstrucción 3D ayudan en la detección de tumores, fracturas y otras anomalías. "El uso de técnicas de PDI en imágenes de resonancia magnética (MRI) mejora la

detección de tumores cerebrales mediante algoritmos de segmentación avanzados" (Chen et al., 2023).

- Vigilancia: se utiliza para reconocimiento facial, seguimiento de personas y análisis de comportamiento en videovigilancia donde "Los algoritmos de reconocimiento facial basados en redes neuronales convolucionales (CNN) alcanzan una precisión superior al 99.8% en condiciones controladas" (Wang et al., 2024).
- Agricultura de precisión: Las imágenes satelitales y de drones procesadas con técnicas de PDI permiten evaluar la salud de los cultivos, por ejemplo: "El análisis espectral de imágenes multiespectrales ayuda a identificar estrés hídrico en plantaciones de maíz" (Botta et al., 2022).
- Automóviles autónomos: Los vehículos autónomos utilizan PDI para detectar peatones, señales de tráfico y obstáculos, por ejemplo: "Los sistemas de visión por computadora en automóviles autónomos combinan PDI con aprendizaje profundo para una navegación segura" (Botta et al., 2018).
- Cartografía automatizada y sistemas de información geográfica SIG: es fundamental para la generación de mapas digitales, actualización de cartografía y análisis espacial, un ejemplo donde "El análisis multitemporal con índices espectrales (NDVI, NDWI) en imágenes Landsat permite monitorear deforestación con un 89% de exactitud" (Tang et al., 2024).

2.2.2 Etapas para el análisis de una imagen

El procesamiento digital de imágenes (PDI) es un flujo estructurado de operaciones que transforma una imagen cruda en información cuantificable y analizable. En la Figura 5 presenta un diagrama general de las principales etapas a considerar para llevar a cabo el análisis de una imagen. A continuación, se describen estas etapas aplicables a la mayoría de campos:

1. Adquisición de la imagen: "La calidad de la imagen adquirida limita el máximo desempeño de cualquier algoritmo de PDI" (González, 2018).
 - a. Resolución espacial: Determinada por el sensor (píxeles/ μm) y óptica.
 - b. Profundidad de bits

- c. Iluminación: Homogénea para evitar sombra.
- 2. Preprocesamiento
 - a. Corrección de artefactos
 - i. Ruido gaussiano
 - ii. Ruido impulsivo
 - b. Mejora de contraste y realce
 - i. Ecualización de histograma
 - ii. Filtros de realce
 - c. Corrección geométrica
 - d. Rotación/recorte
- 3. Segmentación
 - a. Umbralización
 - b. Watershed
 - c. Clusterización (K-means)
 - d. Redes Neuronales (U-Net)
- 4. Extracción de Características
 - a. Morfología
 - b. Textura
 - c. Color
- 5. Análisis
 - a. Clasificación
 - b. Cuantificación
 - c. Visualización
- 6. Almacenamiento
 - a. Formatos sin pérdidas
 - b. Metadatos.

Figura 5

Etapas clave para el procesamiento de imágenes



Nota. Esta ilustración fue tomada de Napkin.

2.2.3 Parámetros de una imagen

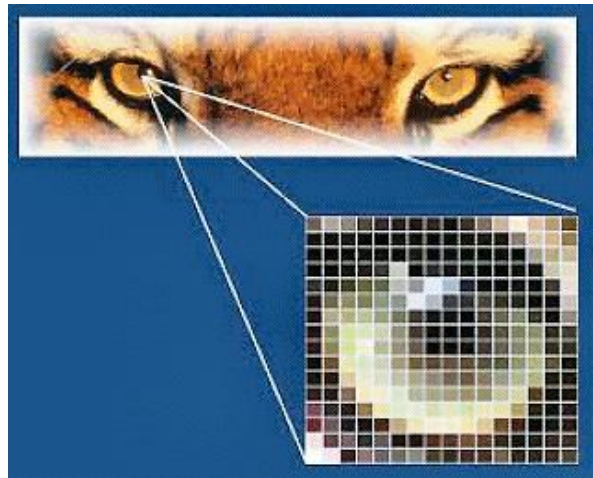
De acuerdo con Querejeta (2017) menciona que una imagen se puede considerar como el conjunto de puntos de colores, es decir una sucesión de puntos que conforman una matriz de información para su implementación digital.

“Una imagen digital está compuesta de un número finito de elementos y cada uno tiene una localidad y un valor particular. A estos elementos se les llama puntos elementales de la imagen o píxeles, siendo este último el término comúnmente utilizado para denotar la unidad mínima de medida de una imagen digital”. En la

Figura 6 ilustra gráficamente la cantidad de píxeles que conforman una imagen (González et al., 2018).

Figura 6

Píxeles que conforman una imagen



Nota. Los parámetros de una imagen digital determinan su calidad, nivel de detalle y aplicabilidad en diferentes campos. Esta ilustración fue tomada de Scielo.

2.2.3.1 Resolución espacial

La resolución espacial se refiere al tamaño del área que representa un píxel en una imagen. En términos simples, es la capacidad de una imagen para mostrar detalles finos. Cuanto menor sea el tamaño del píxel o la distancia de muestra terrestre, mayor será la resolución espacial y, por lo tanto, mayor el detalle (Jensen ,2016). En la Figura 7 ilustra las diferentes resoluciones espaciales que pueden ser consideradas.

Figura 7

Ejemplos de resolución espacial



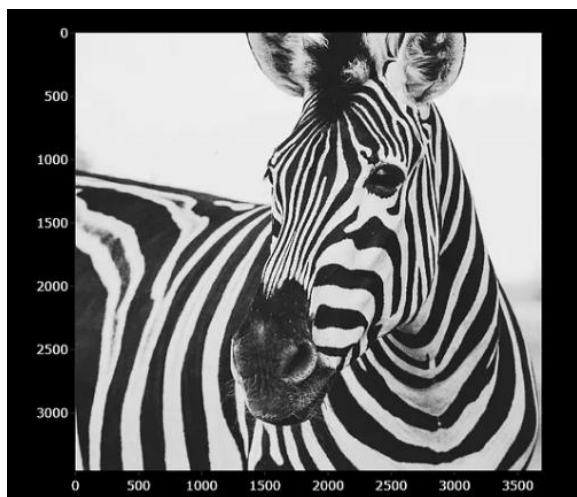
Nota. Esta ilustración fue tomada de EOS Data Analytics.

En el procesamiento digital de imágenes, es fundamental definir claramente la notación utilizada para describir las dimensiones de una imagen. Una imagen bidimensional (2D) se representa comúnmente mediante una matriz de tamaño $M \times N$, donde M representa al alto (número de filas) y N el ancho (número de columnas).

Esta forma de representación es típica para imágenes en escala de grises, donde cada elemento de la matriz contiene un valor de intensidad. En la Figura 8 se muestra la representación visual de una imagen bidimensional en una matriz de tamaño $M \times N$. Por otro lado, una imagen tridimensional (3D) incorpora un tercer componente, representado como $M \times N \times C$, donde C es la profundidad o número de canales.

Figura 8

Ejemplo de una imagen bidimensional representada en una matriz



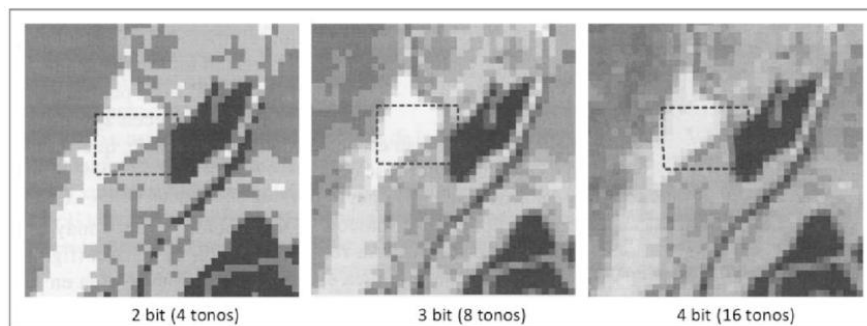
Nota. Esta ilustración fue tomada de Medium.

2.2.3.2 Resolución radiométrica

Según González (2018), "La resolución radiométrica se define por el número de bits utilizados para representar la intensidad de cada píxel, determinando el rango dinámico de la imagen. Un mayor número de bits permite capturar más niveles de intensidad, mejorando la discriminación de detalles sutiles en sombras y áreas brillantes". La Figura 9 muestra los diversos números de bits que pueden ser empleados en una imagen.

Figura 9

Ejemplos de resolución radiométrica.



Nota. Esta ilustración fue tomada de ResearchGate.

Las escalas comunes en imágenes digitales:

- 8 bits: 256 niveles (un ejemplo son las cámaras comunes).
- 12–14 bits: 4,096–16,384 niveles (un ejemplo son los drones profesionales).
- 16 bits: 65,536 niveles (un ejemplo son las imágenes médicas DICOM).

2.2.3.3 Modelos de color

Los modelos de color son sistemas matemáticos que permiten representar colores de manera numérica para su procesamiento digital, cabe destacar que la elección depende de la aplicación. En la Tabla 4 sintetiza las características más importantes y los valores permitidos en cada modelo de color.

Tabla 4

Comparación de modelos de colores más utilizados

Modelo	Descripción	Valores
RGB (Red, Green, Blue)	"El modelo RGB es un sistema aditivo basado en la combinación de tres componentes primarios (rojo, verde y azul) para representar colores en dispositivos de visualización como pantallas y cámaras digitales" (González, 2018).	Utiliza valores entre el rango de 0–255 por canal en 8 bits.
HSV	"El modelo HSV separa la información de color en tres componentes intuitivas: tono (matiz), saturación (pureza del color) y valor (brillo), facilitando operaciones como ajuste de contraste o segmentación por color" (Kumar et al., 2023).	1.- H (Hue): 0°–360° que define el color. 2.- S (Saturation): 0%-100% que va del gris al color puro). 3.- V (Value): 0%-100% que va del negro al color máximo).

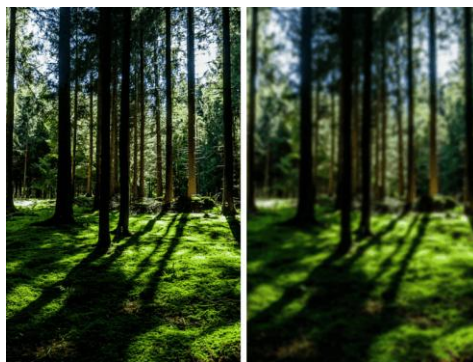
CMYK (Cyan, Magenta, Yellow, Black)	"El modelo CMYK es un modelo sustractivo usado en impresión, donde los colores se generan restando luz mediante tintas. A diferencia de RGB, más tintas oscurecen el resultado" (Phuangsaibai et al, 2021).	Utiliza valores porcentuales entre el 0% al 100% por cada canal.
CIELAB (L*a*b*)	El modelo CIELAB (también llamado La*b*) es un espacio de color perceptualmente uniforme, diseñado para representar colores de manera consistente con la percepción humana, independientemente del dispositivo.	1.- L(Luminosidad): 0% al 100% que va del negro al blanco. 2.-*a* (eje verde-rojo): -128 al 127. 3.-*b* (eje azul-amarillo): -128 al 127

2.2.3.4 Enfoque

El enfoque en una imagen digital se refiere a la nitidez con la que se representan los bordes y detalles finos. En la Figura 10 ilustra gráficamente la comparación de una imagen con diferentes grados de nitidez, donde está directamente relacionado con la frecuencia espacial de la imagen. "Una imagen bien enfocada contiene altas frecuencias espaciales (bordes abruptos), mientras que una imagen desenfocada pierde estos detalles" (González et al., 2018).

Figura 10

Representación de una imagen con diferente grado de nitidez



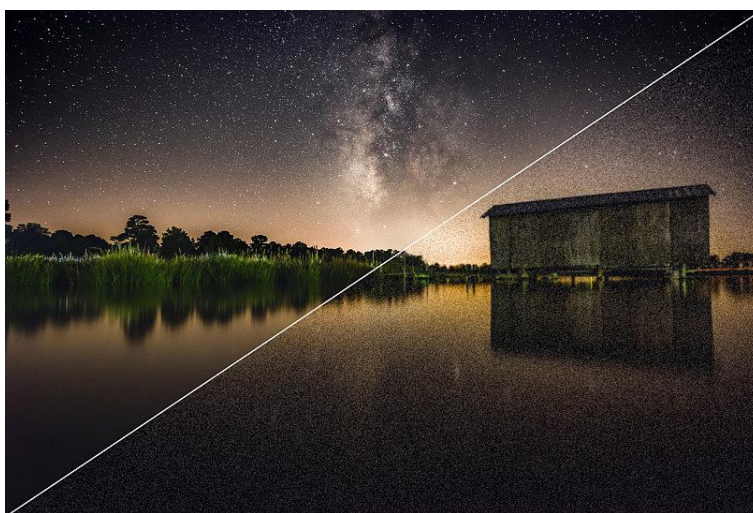
Nota. Esta ilustración fue tomada de Canva.

2.2.3.5 Ruido

De acuerdo con Thornbush (2017), señala que el ruido es “el conjunto de variaciones aleatorias no deseadas que afectan los valores de intensidad de los píxeles, distorsionando la imagen original y reduciendo su calidad. Estas perturbaciones pueden provenir de la captura, transmisión o procesamiento de la imagen”. La Figura 11 muestra una imagen dividida para demostrar la presencia de ruido en una parte de ella.

Figura 11

Ejemplo de ruido en una imagen



Nota. Esta ilustración fue tomada de DZOOM.

2.2.3.6 Contraste

Según Peli (2021), el contraste “es una medida de la diferencia relativa en intensidad entre los objetos de interés y el fondo en una imagen, lo que permite resaltar detalles importantes para su análisis”. Es decir, depende de la luminosidad entre las áreas claras y las sombras de una imagen. En la Figura 12 muestra la diferencia de contraste en una imagen, donde permite identificar la importancia de este debido a que facilita la identificación de los detalles, bordes y elementos dentro de la imagen, en cambio, un contraste reducido puede provocar que la imagen parezca opaca o borrosa.

Figura 12

Ejemplo de contraste en una imagen



Nota. Esta ilustración fue tomada de Photography.

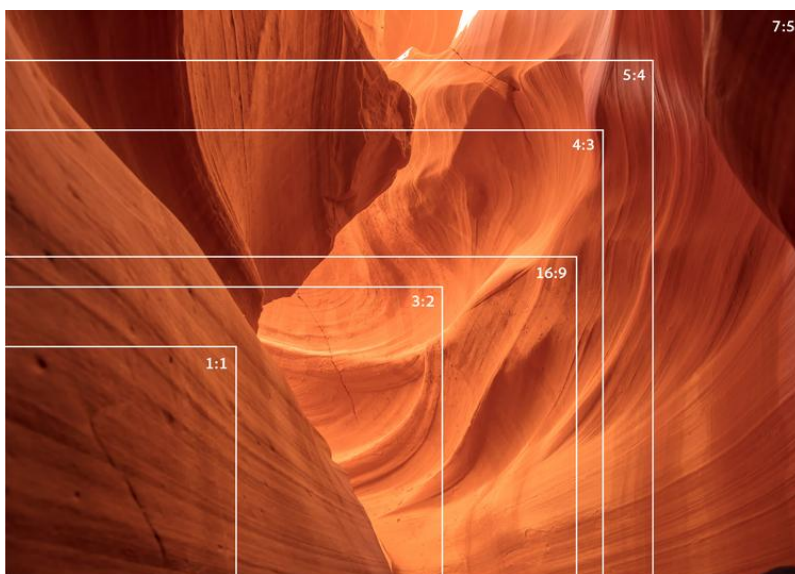
2.2.3.7 Relación de aspecto

La relación de aspecto es la relación proporcional entre el alto y ancho de una imagen digital, sin embargo, las cámaras capturan en distintas relaciones de aspecto, sin embargo, hace referencia a su proporción dimensional, no a la resolución real o al total de píxeles que contiene la imagen. En la Figura 13 muestra todas las relaciones de aspecto utilizadas a lo largo del tiempo en una imagen digital, a continuación, se describen las más empleadas en la actualidad:

- Formato cuadrado [1:1]: Es la relación de algunas cámaras de formato medio. También es compatible con la mayoría de las redes sociales.
- Formato completo [3:2]: Es la relación nativa de muchas cámaras réflex digitales DSLR y aquellas con sensor de recorte APS-C.
- Formato estándar [4:3]: Es la relación que durante mucho tiempo fue el formato predeterminado en televisores y monitores, actualmente es usado en dispositivos móviles.
- Formato panorámico [16:9]: Es la relación estándar actual en la mayoría de dispositivos de video como televisores de alta definición, cámaras de video digitales y plataformas de transmisión de contenido (Adobe, 2024).

Figura 13

Formatos de relación aspecto en una imagen digital



Nota. Esta ilustración fue tomada de Adobe.

2.2.4 Métricas de evaluación de calidad de una imagen

En la Tabla 5 sintetiza las principales métricas de evaluación de calidad en una imagen, donde describe los parámetros considerados y la operación que se realiza para calcularlo.

Tabla 5

Principales parámetros de calidad en una imagen

Parámetro	Operación típica
Resolución espacial	Re muestreo (upsampling / downsampling).
Resolución radiométrica	Ajuste de rango dinámico.
Relación de aspecto	Recorte (cropping) o relleno (padding).
Nitidez	Filtros de realce (Laplaciano, Unsharp Mask).
Ruido	Filtrado (mediana, Gaussiano, NLM).
Contraste	Ecualización de histograma (HE, CLAHE).

2.2.4.1 Relación señal-ruido (SNR)

La Relación Señal-Ruido (SNR) es un parámetro importante en el procesamiento de imágenes que cuantifica la calidad de una imagen al comparar el nivel de la señal útil con el ruido presente. Se expresa comúnmente en decibelios (dB) y es empleado en aplicaciones como imágenes médicas, astronomía y fotografía de baja luminosidad (Igual, 2019).

Como se demuestra en la ecuación 3, la relación señal-ruido se calcula de la siguiente manera:

$$SNR = \frac{\mu_{Señal}}{\sigma_{Ruido}} \quad (3)$$

Donde:

- $\mu_{Señal}$: Es el valor medio de la intensidad en la región de interés.
- σ_{Ruido} : Es la desviación estándar del ruido.

Cuando el SNR tiene un valor alto $> 20dB$ indica que tiene una buena calidad (González & Woods, 2018)

2.3. Visión artificial

Según Microsoft Azure (2025), “Es un campo de la informática que se centra en permitir que los equipos identifiquen y comprendan objetos y personas en imágenes y vídeos. Al igual que otros tipos de inteligencia artificial, el objetivo de la visión artificial es realizar y automatizar tareas que replican las capacidades humanas”. Para ello emplea algoritmos, cámaras y procesamiento digital en lugar de los órganos y procesos biológicos de la visión humana.

2.3.1 Fundamentos de visión artificial

“La visión artificial necesita una gran cantidad de datos. Ejecuta análisis de datos una y otra vez hasta que discierne distinciones y finalmente reconoce imágenes. Por ejemplo, para entrenar a una computadora para que reconozca llantas de automóvil, necesita recibir grandes cantidades de imágenes de llantas y elementos

relacionados con llantas para aprender las diferencias y reconocer una llanta, especialmente una sin defectos” (IBM, 2021).

2.3.2 Principales aplicaciones de la visión artificial

La visión artificial tiene un impacto en múltiples sectores:

- Industria: El control de calidad automatizado con el fin de inspeccionar productos en líneas de ensamblaje.
- Robótica: Manipulación de objetos.
- Medicina: Proporciona un diagnóstico asistido y cirugía robótica.
- Automóviles autónomos: Detección de obstáculos y reconocimiento de señales de tráfico.
- Seguridad y vigilancia: Reconocimiento facial y búsqueda de personas.
- Agricultura de precisión: Monitoreo de cultivos.
- Entretenimiento: Realidad aumentada (AR) y Generación de imágenes.

2.3.3. Técnicas comunes en visión artificial

En los últimos años, la visión artificial ha evolucionado gracias al avance de diversas técnicas de inteligencia artificial, debido a que estas herramientas permiten mejorar la precisión en tareas como clasificación de imágenes, detección de objetos y segmentación semántica. A continuación, se describen algunas de las técnicas más relevantes.

2.3.3.1 Aprendizaje automático (ML)

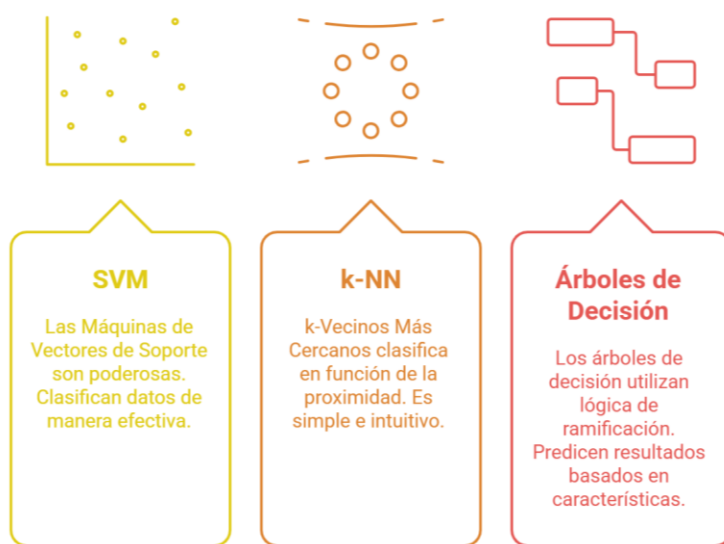
El aprendizaje automático es el campo de estudio que proporciona a los equipos informáticos la capacidad de aprender sin ser programados de manera explícita, es decir se emplea para enseñar a las máquinas la manipulación y manejo de datos de forma eficiente, debido a que en algunos escenarios no es posible llevar a cabo interpretación de dicha información que se extraen de estos, ahí su implementación. Debido a la gran cantidad de datos disponibles, la demanda de aprendizaje automático va en crecimiento ya que es aplicado en múltiples áreas de la industria

para extraer datos relevantes, se podría considerar que el objetivo principal del aprendizaje automático es aprender de datos.(Mahesh, 2020).

En visión artificial, se utiliza para tareas como clasificación de imágenes, detección de anomalías y predicción de características visuales. Algoritmos como Support Vector Machines (SVM), k-Nearest Neighbors (k-NN) y árboles de decisión han sido tradicionalmente empleados en este campo (Goodfellow, et al., 2016). En la Figura 14 ilustra de manera grafica una breve descripción de cada algoritmo previamente mencionado.

Figura 14

Principales algoritmos de aprendizaje automático



Nota. Esta ilustración fue tomada de Napkin.

2.3.3.2 Redes Neuronales Convolucionales (CNN)

Las Redes Neuronales Convolucionales (CNN) es considerada una arquitectura especializada en el procesamiento de datos. Han demostrado gran eficacia en tareas como reconocimiento facial, clasificación de objetos y segmentación de imágenes. Su funcionamiento se basa en capas convolucionales que extraen características espaciales de manera jerárquica (Goodfellow et al., 2023).

Las capas convolucionales se podrían considerar como un conjunto de filtros llamados campos receptivos, ya que estos se ajustan para la manipulación y extracción de características de una señal. En contraposición a una red neuronal tradicional, en la que cada neurona de una capa se conecta a todas las neuronas de la otra, conocido como Fully Connected (FC), en las CNN se comparten las neuronas a través de filtros que permiten extraer información de las imágenes de entrada (Calcagni., 2020).

2.3.3.3 Redes Generativas (GANs)

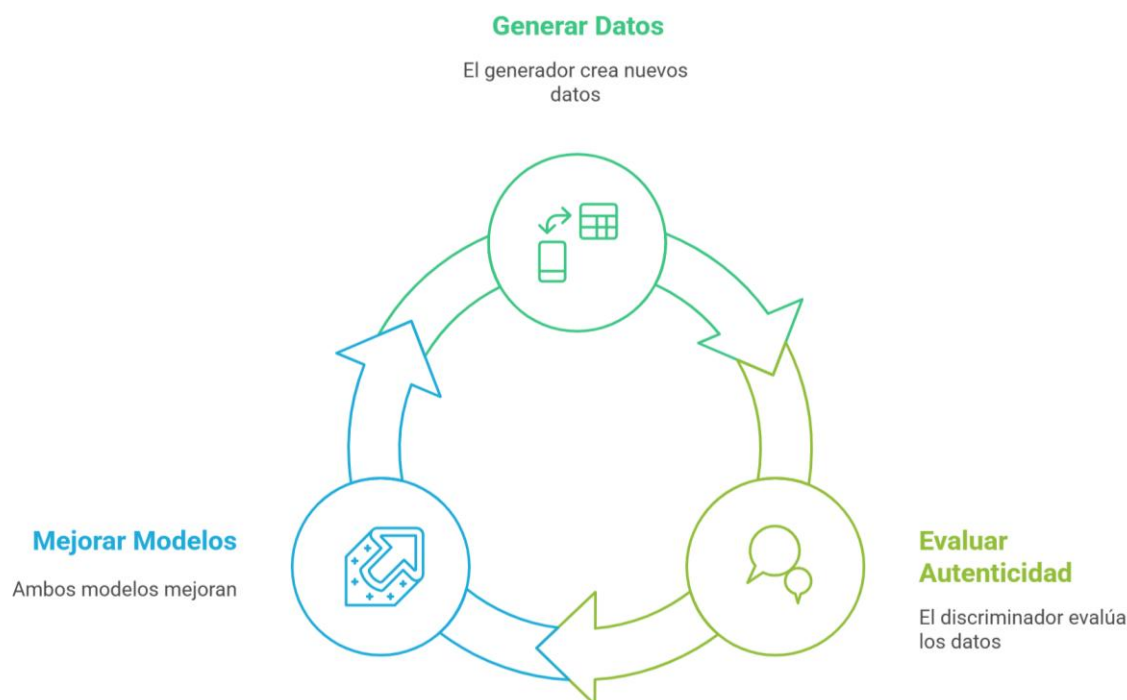
Según Wang et al (2022), las redes generativas antagónicas se componen de dos modelos: el generador y el discriminador, entrenados al mismo tiempo para desafiarse uno al otro.

En la Figura 15 muestra el ciclo general de entrenamiento para las redes generativas, donde el generador es capacitado para producir datos falsos que sean lo más similares posibles a los ejemplos reales de un entrenamiento seleccionado, mientras que el discriminador es entrenado para ser capaz de distinguir los datos falsos producidos por el generador de aquellos que corresponden al conjunto de entrenamiento.

Así mismo, ambos modelos intentan de manera constante superarse, cuanto mayor sea la capacidad del generador para el desarrollo de datos convincentes, mayor habilidad tendrá el discriminador para distinguir los ejemplos auténticos de los falsos. (Calcagni., 2020).

Figura 15

Ciclo de entrenamiento de redes generativas



Nota. Esta ilustración fue tomada de Napkin.

2.3.3.4 Transformers (ViT)

Los Transformers han transformado el proceso de procesamiento de lenguaje natural y recientemente se han ajustado a la visión artificial a través de arquitecturas como los Vision Transformers (ViT). Estas redes sustituyen las convoluciones por mecanismos de concentración, lo que facilita la modelación de relaciones a nivel mundial entre píxeles. ViT ha evidenciado un desempeño competitivo en labores de categorización y segmentación de imágenes, en particular cuando se cuenta con grandes cantidades de datos para entrenamiento (Dosovitskiy et al., 2021).

2.3.4. Métricas de rendimiento

El rendimiento del modelo indica qué tan bien un modelo de machine learning (ML) lleva a cabo la tarea para la que fue diseñado, basado en diversas métricas. El rendimiento de un modelo de IA generalmente se mide mediante un conjunto de

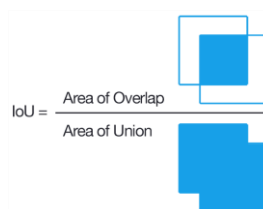
pruebas, comparando los resultados del modelo con las predicciones en el conjunto de pruebas de referencia. Los insights obtenidos de la evaluación del rendimiento ayudan a determinar si un modelo está listo para el despliegue en el mundo real o si necesita ajustes o capacitación adicional (AWS, 2025).

2.3.4.1 Intersección sobre la unión

El IoU es una medida que varía entre 0 y 1 y mide el nivel de solapamiento entre dos cajas delimitadoras. La Figura 16 muestra cómo se calcula este índice: el área de superposición es la zona donde la caja predicha por el modelo y la caja delimitadora real se cruzan, y el área de unión es igual al área total cubierta por las dos cajas, sin incluir el área que ambas comparten. Este valor permite identificar qué tan precisas son las predicciones del modelo cuando se realiza la detección de objetos (Martínez et al., 2023).

Figura 16

Cálculo de IoU en una caja delimitadora.



Nota. Esta ilustración fue tomada de PyImageSearch.

En la Figura 17 muestra los valores estimados de IoU. Un valor bajo indica que la caja delimitadora predicha no cubre adecuadamente el objeto de interés, mientras que un valor alto refleja que la predicción coincide con mayor precisión con el área real del objeto.

Figura 17

Valores aproximados del IoU.



Nota. Esta ilustración fue tomada de PylImageSearch.

2.3.4.2 Matriz de confusión

La matriz de confusión es una herramienta que permite visualizar el desempeño de un modelo al predecir cada clase dentro del conjunto de datos de prueba. (Cotrina et al., 2025).

Está conformado por los siguientes:

- Verdadero Positivo (True Positive, TP): Se produce cuando el modelo ha detectado un objeto y IoU es mayor al umbral de detección, considerándose una predicción correcta.
- Falso Positivo (False Positive, FP): Se produce cuando el modelo detecta un objeto, pero al calcular el IoU es menor al umbral de detección, es decir se trata de una detección incorrecta.
- Falso negativo (False Negative, FN): Se produce cuando el modelo predice incorrectamente un resultado negativo cuando la realidad es positiva.
- Verdadero Negativo (True Negative, TN): Indican la probabilidad de que un valor sea negativo si resultó negativo en la predicción.

En la Figura 18 muestran los escenarios posibles entre las clases reales y las predichas. Las clases 1 y 2 están representadas por los cuadros de color azul y naranja, que corresponden a los aciertos (Verdaderos positivos). Por otra parte, los cuadros amarillos y verdes simbolizan los errores, es decir el modelo clasifica de manera equivocada una instancia de una clase como perteneciente a la otra.

Figura 18

Matriz de confusión.

	Predicción Clase 1	Predicción Clase 2
Valor real Clase 1	Aciertos True Positive Clase 1	Fallos False Positive Clase 2
Valor real Clase 2	Fallos False Positive Clase 1	Aciertos True Positive Clase 2

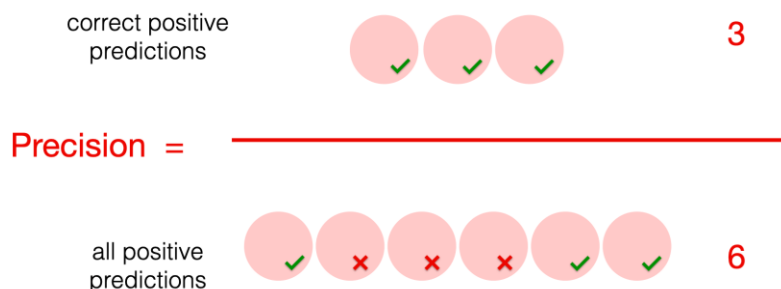
2.3.4.3 Precisión

Es el porcentaje de predicciones correctas realizadas por el modelo entrenado, en los términos más simples, la precisión es la relación entre los verdaderos positivos y todos los positivos. Cabe mencionar que este valor es dependiente del umbral de IoU especificado, lo que significa que este puede mejorar o empeorar de acuerdo a dicho umbral (Martínez et al., 2025).

En la Figura 19 ilustra la manera de calcular la precisión donde se divide el número de predicciones positivas correctas (verdaderos positivos) entre el número total de instancias que el modelo predijo como positivas (tanto verdaderos como falsos positivos).

Figura 19

Cálculo de la precisión.



Nota. Esta ilustración fue tomada de EvidentlyAI.

2.3.4.4 Recall

El Recall o sensibilidad es una medida que permite conocer la proporción de casos positivos que fueron correctamente clasificados. En un modelo perfecto el recall es igual a 1 para cada clase. Desde el punto de vista analítico un investigador busca aumentar el recall sin afectar el valor de la accuracy (Borja et al., 2020).

De acuerdo con Saji. (2019).” La precisión se centra en que tan correctas son las detecciones, mientras que el recall se centra en la completitud de las detecciones”.

En la Figura 20 ilustra la manera de calcular el recall, dividiendo el número de verdaderos positivos entre el número de casos positivos. Este último incluye los verdaderos positivos (casos identificados correctamente) y los falsos negativos (casos no detectados).

Figura 20

Cálculo del recall.



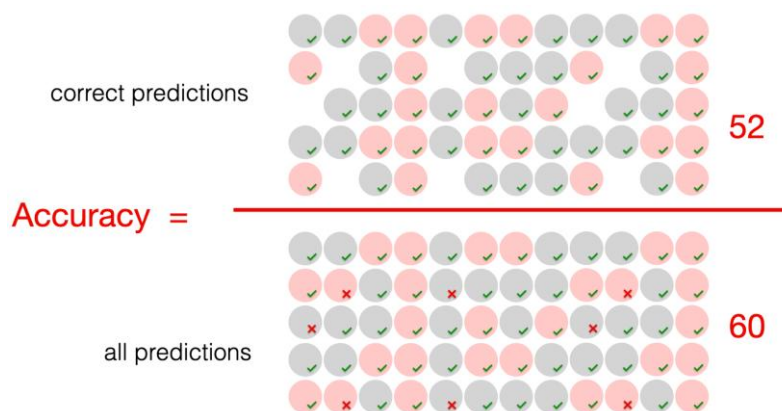
Nota. Esta ilustración fue tomada de EvidentlyAI.

2.3.4.5 Accuracy

Es una métrica que mide la frecuencia con la que un modelo de aprendizaje automático predice correctamente el resultado. En la Figura 21 ilustra como se puede calcular dividiendo el número de predicciones correctas entre el número total de predicciones (Fleuren, 2020).

Figura 21

Cálculo de accuracy.



Nota. Esta ilustración fue tomada de EvidentlyAI.

2.3.4.6 Precisión media (mAP)

Se calcula obteniendo al valor promedio de precisión para cada clase y luego obteniendo la media de la precisión en todas las clases cuando el umbral IoU. mAP50-95 se obtiene de la misma manera con el único cambio siendo que cuenta con múltiples umbrales de IoU desde 0,50 hasta 0,95, con valores tomados cada 0.05, dando un total de 10 valores, donde luego se toma el valor promedio. Cuanto mayor sea el indicador, mejor será la detección del modelo (Lui et al., 2022).

2.4 Tecnologías utilizadas en EasyCountCol

Las herramientas tecnológicas aluden a todo dispositivo o software cuyo propósito es aportar al progreso del proyecto.

2.4.1 Lenguajes de programación

Los lenguajes de programación son esenciales para el desarrollo de programas informáticos, el análisis de datos, la automatización y las aplicaciones. Su selección se basa en el objetivo, eficacia y sencillez para su utilización. En la Tabla 6 presenta una comparativa entre tres lenguajes de programación (Python, Dart y SQL) que fueron utilizados, destacando sus principales usos y ventajas. A continuación, se describe su contenido:

Tabla 6

Principales usos de los lenguajes seleccionados

Lenguaje	Usos principales	Ventajas
Python	Ciencia de datos, desarrollo web, automatización, scripting e inteligencia artificial.	Amplia comunidad, librerías y fácil aprendizaje.
Dart	Desarrollo móvil con flutter para iOS, Android y desarrollo web.	Rendimiento eficiente gracias a su compilación y sintaxis similar a Java.
SQL	Consulta, manipulación de datos, diseño y administración de base de datos.	Estandarizado, ampliamente utilizado y eficiencia en grandes volúmenes de datos.

2.4.1.1 Python

“Es un lenguaje de programación interpretado, interactivo y orientado a objetos. Incorpora módulos, excepciones, tipado dinámico, tipos de datos dinámicos de alto nivel y clases. Admite múltiples paradigmas de programación más allá de la programación orientada a objetos, como la programación procedimental y funcional” (Python, 2024).

“El lenguaje incluye una amplia biblioteca estándar que abarca áreas como el procesamiento de cadenas (expresiones regulares, Unicode, cálculo de diferencias entre archivos), protocolos de internet (HTTP, FTP, SMTP, XML-RPC, POP, IMAP), ingeniería de software (pruebas unitarias, registro, creación de perfiles, análisis de código Python) e interfaces de sistemas operativos (llamadas al sistema, sistemas de archivos, sockets TCP/IP)” (Python, 2024).

2.4.1.2 Dart

Se trata de un lenguaje de programación diseñado para crear interfaces de usuario modernas, ágiles y multiplataforma. Fue creado por Google y se le reconoce como un lenguaje orientado a objetos. Además, permite la compilación previa y la ejecución en directo. Una de las aplicaciones más destacadas de Dart es la creación de aplicaciones para móviles a través del framework Flutter. (Dart,2025).

2.4.1.3 SQL

Según Melton (2002), describe que es un lenguaje de programación estándar empleado para interactuar con las bases de datos relacionales. Fue desarrollado en 1970, desde entonces ha sido utilizado en múltiples sistemas de gestión de bases de datos como lo son: MySQL, PostgreSQL, Microsoft SQL Server, Oracle y MariaDB.

Este permite establecer la estructura de una base de datos, gestionar los datos, además de poseer la capacidad de realizar consultas complejas, facilitando la búsqueda y organización de volúmenes de información

2.4.2 Bibliotecas y frameworks de visión artificial

Los frameworks y bibliotecas de visión artificial ofrecen instrumentos fundamentales para el procesamiento, análisis y entendimiento de imágenes digitales a través de algoritmos informáticos. Estas tecnologías facilitan la automatización de labores tales como la identificación de patrones, la cuantificación de objetos, la división de imágenes y la categorización de datos visuales.

2.4.2.1 TensorFlow

Es un framework de código abierto desarrollado por Google con el objetivo de implementar el aprendizaje automático y la computación numérica, tomando como enfoque principal el desarrollo de redes neuronales profundas, cuenta además con módulos especializados y bibliotecas complementarias como TensorFlow Lite y TensorFlow Hub que permiten la creación de modelos para la clasificación e identificación de objetos en una imagen (TensorFlow, 2025).

Debido a su soporte y escalabilidad para hardware acelerado (GPU y TPUs), este framework se utiliza ampliamente en aplicaciones que requieren el procesamiento visual complejo, como reconocimiento de imágenes.

2.4.2.2 PyTorch

Es un marco completo para crear modelos de aprendizaje profundo, se utiliza habitualmente en aplicaciones como el reconocimiento de imágenes y el

procesamiento del lenguaje. Se distingue por su gran compatibilidad con las GPU y su uso de la auto diferenciación en modo inverso, que permite modificar los gráficos de cálculo sobre la marcha. Combina las bibliotecas backend aceleradas por GPU y flexibles de Torch con una interfaz Python intuitiva que se centra en la creación rápida de prototipos, el código legible y la compatibilidad con la mayor variedad posible de modelos de aprendizaje profundo (NVIDIA, 2025).

2.4.2.3 YOLO

Es una sola red convolucional que predice simultáneamente múltiples cuadros delimitadores y probabilidades de clase para esos cuadros. Es decir, usa las características de toda la imagen para predecir cada cuadro delimitador. También predice todos los cuadros delimitadores en todas las clases para una imagen simultáneamente. Este divide la imagen de entrada en una cuadrícula $S \times S$. Si el centro de un objeto se ubica en una celda de la cuadrícula, esa celda es responsable de detectar ese objeto. Se entrena con imágenes completas y optimiza directamente el rendimiento de detección (Pérez et al., 2019).

De acuerdo con Ramos (2021), el modelo usa Deep learning y CNN en la detección de objetos, y se caracteriza por “Ver” una sola vez la imagen, lo que acelera el resultado final, cuando se hace la división de una cuadrícula $S \times S$ en cada celda predice N posibles “cuadros delimitadores”, calculando el nivel de probabilidad, realizando el cálculo $S \times S \times N$ en gran parte con un nivel de incertidumbre bajo. Cuando se obtiene las predicciones se eliminan las cajas que están por debajo del límite, y las que restan pasan por “supresión no máxima”, que sirve para eliminar los posibles elementos que se detectan como duplicados, de esta manera queda el más exacto.

Cada cuadro delimitador consta de 5 predicciones: x , y , w , h , y la confianza. Las coordenadas (x ; y) representan el centro del cuadro en relación con los límites de la celda de la cuadrícula (Rozada, 2021).

2.4.2.4 OpenCV

Open Source Computer Vision Library es una librería de código abierto empleada para el desarrollo de aplicaciones de visión computacional en tiempo real, debido a que ofrece algoritmos optimizados para operaciones básicas de procesamiento de imágenes hasta la detección de objetos, reconocimiento facial y análisis de movimientos (OpenCV, 2025).

Además de su compatibilidad con múltiples lenguajes de programación como Python, C++ y Java, OpenCV es multiplataforma, lo que permite su implementación en sistemas operativos como Windows, macOS, Linux, Android e iOS.

2.4.2.5 Pillow

Es una biblioteca de procesamiento de imágenes para Python que actúa como una versión mejorada de la librería PIL, es ampliamente utilizada para tareas básicas como abrir, modificar, convertir y guardar imágenes en distintos formatos. Permite realizar operaciones como redimensionamiento, rotación, ajustes y conversión de color, siendo empleada para realizar el preprocesamiento de imágenes. (Pillow, 2025).

En la Tabla 7 se describen las principales aplicaciones y ventajas de tres librerías o frameworks ampliamente utilizados en el procesamiento de imágenes.

Tabla 7
Principales aplicaciones y ventajas de librerías utilizadas para el procesamiento de imágenes

Librería / Framework	Aplicaciones	Ventajas
OpenCV	Procesamiento de imágenes, detección de rostros, objetos, realidad aumentada y calibración de cámaras.	Amplia documentación y comunidad además de optimizada para rendimiento en tiempo real

TensorFlow	Entrenamiento de modelos de redes neuronales convolucionales CNN, clasificación y segmentación de imágenes e implementación en dispositivos móviles.	Escalabilidad en proyectos grandes, además de integración con Keras.
Pillow	Apertura, guardado, conversión de formatos y operaciones básicas con imágenes.	Sintaxis sencillas para tareas rápidas además de compatibilidad con OpenCV.
YOLO	Detección de objetos en tiempo real para la vigilancia, conducción autónoma, análisis industrial, medicina, retail y sistemas móviles inteligentes.	Extremadamente rápido al procesar imágenes porque detecta objetos en una sola pasada de la red.
PyTorch	Construir y entrenar redes neuronales además de permitir un desarrollo rápido gracias a su computación dinámica.	Dinámico y flexible para construir modelos gracias a su dynamic computation graph,

2.4.3 Otras herramientas tecnológicas

El desarrollo móvil consiste en la creación de aplicaciones para dispositivos como smartphones y tabletas. Este proceso implica el uso de frameworks, lenguajes de programación y herramientas especializadas que permiten diseñar interfaces intuitivas y funcionalidades robustas. Además de los lenguajes tradicionales, existen otras herramientas tecnológicas que complementan el desarrollo móvil y facilitan la integración de componentes como bases de datos, visión artificial, conectividad con sensores y análisis en tiempo real, lo cual resulta esencial para proyectos como EasyCountCol.

2.4.3.1 Flutter

Es un framework de código abierto desarrollado por Google para el desarrollo de aplicaciones nativas multiplataforma, en diferentes sistemas operativos como lo con

iOS, Android, web y desktop, tomando en cuenta una única base de código. Para ello se emplea el lenguaje Dart. Ofrece una la personalización de interfaces de alta calidad mediante widgets.

Sus características principales son las siguientes:

1. Hot Reload: Permite ver cambios en tiempo real sin reiniciar la app.
2. Widgets propios: Diseño adaptable con Material Design (Android) y Cupertino (iOS).
3. Rendimiento nativo: Compilación a código nativo (ARM, x86) mediante el motor Skia.

Las ventajas de su implementación:

1. Desarrollo rápido y multiplataforma.
2. Amplia documentación y comunidad.
3. Ideal para prototipado y MVP (Google, 2025).

2.4.3.2 Sistemas operativos en dispositivos móviles

De acuerdo con Morley et al. (2021), un sistema operativo móvil es un programa que sirve de enlace entre el hardware de un dispositivo móvil (como un teléfono inteligente o una tableta) y las aplicaciones que se encuentran en su interior. Su tarea primordial consiste en administrar los recursos del dispositivo (procesador, memoria, almacenamiento, sensores, conectividad, etc.) y ofrecer una interfaz que permita al usuario interactuar con el sistema y las aplicaciones.

2.4.3.2.1 Android

Es un sistema operativo móvil de código abierto basado en el kernel de Linux, desarrollado inicialmente por Android Inc, que posteriormente fue adquirido por Google. Es el sistema operativo móvil más utilizado a nivel mundial, con presencia en smartphones, tablets, wearables, televisores y otros dispositivos inteligentes.

Arquitectura técnica:

- Núcleo Linux: Proporciona gestión de memoria, seguridad y control de dispositivos.

- Runtime de Android (ART): Entorno de ejecución optimizado que compila bytecode DEX a código máquina.
- Marco de trabajo de aplicaciones: Ofrece APIs para desarrollo de apps en Java/Kotlin (Android Open-Source Project, 2023).

2.4.3.2.2 IOS

Es un sistema operativo desarrollado por Apple para dispositivos móviles, como lo son el iPhone y el iPad. Sin embargo, es un sistema cerrado, lo que le permite ofrecer un entorno regulado y muy perfeccionado en términos de rendimiento, seguridad y estabilidad (Apple, 2023).

Las aplicaciones para iOS se desarrollan en lenguajes como Swift, aunque también es posible integrarlas mediante frameworks multiplataforma, gracias a su ecosistema regulado y su vinculación con los dispositivos de Apple han convertido a iOS en una plataforma para desarrollos orientados a usuarios que valoran la experiencia de alta gama y la privacidad.

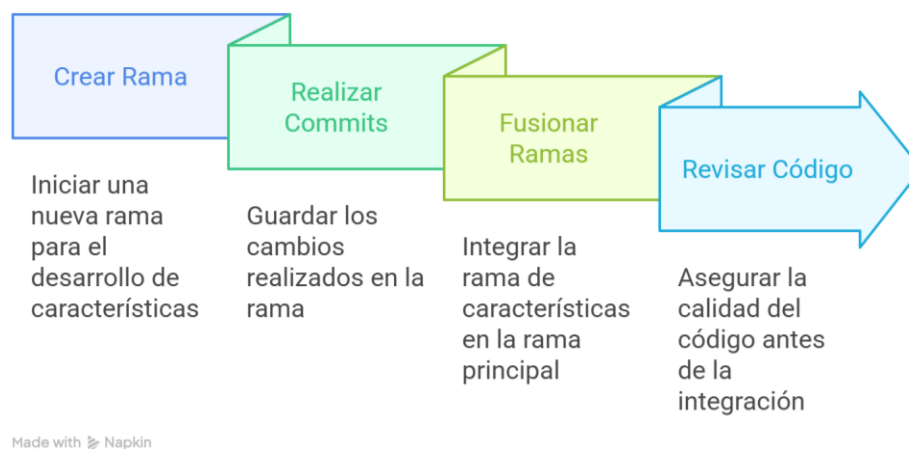
2.4.4.1 Git

De acuerdo con Straub (2021), “Git es un sistema de control de versiones distribuido disponible en todas las plataformas de desarrollo convencionales mediante una licencia de software libre”, este prioriza las revisiones del software además de proporcionar a cada desarrollador una copia privada completa del repositorio de software y numerosas formas de gestionar las revisiones dentro de su contexto.

Además, permite crear diversos flujos de trabajo distribuidos, la mayoría de los cuales son imposibles de ejecutar en un sistema de control de versiones centralizado tradicional. En la Figura 22 ilustra el flujo de trabajo en git, este comienza desde el inicio de una rama hasta la revisión de código.

Figura 22

Flujo de trabajo en Git



Nota. Esta ilustración fue elaborada de Napkin.

2.4.4.2 Github

GitHub es un servidor de alojamiento para almacenar los proyectos basados en Git donde permite la colaboración entre múltiples usuarios, donde se emplea el termino de repositorio, se podría describir como un directorio donde permite el desarrollo de un proyecto, es decir contiene todos los archivos necesarios para el mismo. Aunque existen distintos repositorios remotos con funcionalidades similares, GitHub actualmente es el más empleado, una vez registrado el proyecto permite de manera remota compartirlo entre distintos usuarios y proporciona la seguridad de la nube entre otras funciones (Astigarraga, 2022).

2.4.4.3 Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente creado por Microsoft, que fusiona sencillez con características poderosas características de un ambiente de desarrollo integrado (IDE). Permite el uso de diversos lenguajes de programación como Python, JavaScript, Dart, entre otros, a través de extensiones.

Ofrece características como el resaltado de sintaxis, depuración integrada, gestión de versiones con Git, terminal integrada y una amplia comunidad que proporciona miles de extensiones que potencian su funcionalidad (Microsoft, 2025).

2.4.4.4 Android Studio

Android Studio es el editor de código fuente oficial para la creación de aplicaciones móviles para Android, propone potenciar la productividad mediante Gemini, un asistente de programación propulsado por Inteligencia Artificial. Fue creado por Google y está desarrollado para las aplicaciones en el sistema operativo Android. Este incluye herramientas específicas como el emulador de dispositivos, el creador de interfaces gráficas, instrumentos de depuración y evaluación del rendimiento, permite la codificación en Java, Kotlin y, mediante la integración con Flutter, en Dart (Android Developers, 2025).

2.4.5 APIs REST

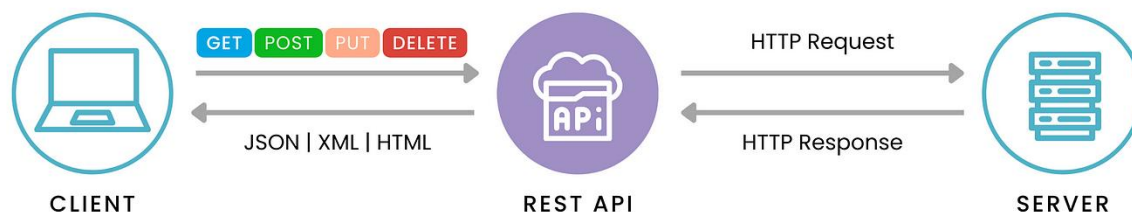
Una API REST (Interfaz de Programación de Aplicaciones Representational State Transfer) es un conjunto de normas que permiten y facilitan la comunicación entre programas mediante el uso del protocolo HTTP. Los elementos que conforma este modelo son los siguientes:

- Cliente: Programa o aplicación que envía solicitudes HTTP.
- Servidor: Programa o servicio que procesa las peticiones y devuelve respuestas.
- Recurso: La entidad que se representa mediante una URL.
- Método: La operación que requiere realizar, siendo las siguientes (GET, POST, PUT, DELETE).

Según Richardson (2021), “REST es un estilo arquitectónico que utiliza métodos HTTP estándar y URIs bien definidos para exponer recursos y permitir su manipulación”. En la Figura 23 muestra el funcionamiento básico de una API REST, ilustrando la interacción entre un cliente, la API y el servidor.

Figura 23

Diagrama de funcionamiento de una API REST



Nota. Esta ilustración fue tomada de SkipLevel.

2.4.5.1 FastAPI

FastAPI es un framework para la creación de APIs en Python, creado para utilizar las propiedades de tipado estático y asincronía del lenguaje. Se distingue por su excelente desempeño y creación automática de documentos a través de Swagger y ReDoc (Ramírez, 2024).

2.4.5.2 Uvicorn

Uvicorn es un servidor web asequible y asincrónico que implementa con ASGI, fue principalmente diseñado para operar aplicaciones modernas en Python con FastAPI. Hace uso de uvloop y httptools con el fin de obtener un desempeño superior en comparación con servidores WSGI convencionales como Gunicorn o uWSGI. Se emplea a menudo en implementaciones de APIs REST ya que soporta eficazmente conexiones simultáneas, lo que lo convierte en perfecto para microservicios y sistemas con gran volumen de tráfico. (Uvicorn, 2024).

2.4.5.3 Docker

Es una plataforma de código abierto que ejecuta aplicaciones y hace que el proceso sea más sencillo al momento de desarrollar y distribuir. Las aplicaciones que se construyen en docker se empaquetan con todas las dependencias de apoyo en una forma estándar llamado un contenedor. Gracias a ello proporciona una facilidad para automatizar las aplicaciones cuando se despliegan. En un entorno donde las

aplicaciones son virtualizadas y ejecutadas, docker añade una capa extra de motor de despliegue (Rad et al., 2018).

2.5 Base de datos

De acuerdo con Date (2023), “una base de datos es una colección de información organizada y presentada para servir a un propósito específico”, también se puede interpretar como un archivo de datos relacionados y recolectados con el fin de satisfacer las necesidades de información de un usuario.

2.5.1 Sistema gestor de base de datos

“Un sistema gestor de bases de datos sus siglas en inglés Data Base Management System (DBMS) se define como una colección de datos interrelacionados y un conjunto de programas para acceder a dichos datos. Este permite almacenar y recuperar la información de una BD de manera práctica y eficiente” (Juárez, P., & Ivonne, I., 2018).

2.5.2 Base de datos relacionales

Una base de datos relacional (RDB) es un tipo de base de datos que estructura la información en tablas, las cuales están compuestas por filas y columnas, estas tablas pueden estar relacionadas entre sí mediante claves primarias y claves foráneas, gracias a ello permite organizar los datos de manera lógica y consistente (Navathe et al., 2018).

Los ejemplos de bases de datos relacionales más famosos son: MySQL, MySQL Server y MariaDB.

2.5.2.1 MySQL

MySQL es un sistema de administración de bases de datos relacional, desarrollado inicialmente por la empresa MySQL AB y actualmente gestionado por Oracle Corporation. Este se fundamenta en el lenguaje SQL (Structured Query Language) para administración, consulta y edición de los datos almacenados en tablas relacionales.

Este sistema es ampliamente empleado en programas web y corporativos debido a su estabilidad, eficiencia y facilidad de integración con múltiples lenguajes de programación como PHP, Python y Java (Dubois, 2021).

2.5.2.2 MariaDB

Según Urtiaga (2020), MariaDB es un sistema de gestión de bases de datos relacionales que proviene de MySQL, desarrollado por la comunidad de software libre como una bifurcación de este sistema previo. Se caracteriza por su total compatibilidad con MySQL, tanto en términos de estructura como de comandos, además de incluir mejoras en rendimiento, protección y almacenamiento que se han implementado de forma activa desde su inicio. Realiza operaciones como las siguientes:

- Integridad de referencia
- Procedimientos almacenados
- Emplea diversos motores de almacenamiento como InnoDB y Aria.

2.5.3 Base de datos no relacionales

Una base de datos no relaciona (NoSQL) es un modelo que no emplea el uso de tablas, su principal propósito es la gestión de grandes cantidades de datos no estructurados o semiestructurados, su aplicación es apropiada para sistemas modernos en tiempo real y big data. Estas ofrecen flexibilidad y escalabilidad en ciertos escenarios donde las bases de datos relacionales no son eficientes (Sadalage, P. J., & Fowler, M, 2024)

2.5.3.1 MongoDB

MongoDB es un sistema orientado a documentos para la administración de bases de datos NoSQL, desarrollado por la empresa MongoDB Inc, entre sus funciones permite la escalabilidad, la flexibilidad y un modelo de consultas e indexación avanzada. Este almacena sus datos en documentos tipo JSON o BSON, cada

documento representa un registro y contiene estructuras complejas como subdocumentos o listas anidadas para moldear los datos (MongoDB Inc, 2024).

2.6 Computo en la nube

Según el National Institute of Standards and Technology (2011), el cómputo en la nube es: "un modelo para permitir el acceso ubicuo, conveniente y bajo demanda a un conjunto compartido de recursos de computación configurables (por ejemplo, redes, servidores, almacenamiento, aplicaciones y servicios) que pueden ser rápidamente aprovisionados y liberados con una mínima gestión o interacción con el proveedor de servicios."

2.6.1 Principales modelos de servicio

Los principales modelos de servicio en la nube son: IaaS, PaaS y SaaS.

2.6.1.1 Infraestructura como servicio (IaaS)

La Infraestructura como servicio (IaaS) ofrece recursos de computación virtualizados a través de internet. En este modelo, el proveedor de servicios proporciona máquinas virtuales, redes, almacenamiento y otros recursos de infraestructura para que el usuario cuente con la capacidad de configurar y administrar los recursos para cumplir sus requerimientos y necesidades. IaaS es ideal para programadores o compañías que quieren tener más dominio sobre el sistema operativo y las aplicaciones (Rittinghouse et al., 2017).

Algunos ejemplos son: Amazon EC2 (AWS), Google Compute Engine, Microsoft Azure Virtual Machines.

2.6.1.2 Plataforma como servicio (PaaS)

De acuerdo con Google Cloud (2025), es una plataforma como servicio (PaaS) "es un modelo de servicio de computación en la nube que ofrece una plataforma flexible y escalable para desarrollar, implementar, ejecutar y administrar aplicaciones. PaaS proporciona todo lo necesario para el desarrollo de aplicaciones, sin las complicaciones de actualizar el sistema operativo y las herramientas de desarrollo, ni el mantenimiento del hardware. En su lugar, un proveedor de servicios externo proporciona todo el entorno PaaS (o plataforma) a través de la nube".

Algunos ejemplos son: Google App Engine, Heroku, AWS Elastic Beanstalk.

2.6.1.3 Software como servicio (SaaS)

Según Microsoft Azure (2025), el software como servicio SaaS “es un modelo de distribución de software basado en la nube donde las personas u organizaciones se suscriben a las aplicaciones en lugar de comprarlas e instalarlas localmente. Los clientes acceden al software a través de internet, generalmente mediante un navegador web, mientras que el proveedor de servicios en la nube gestiona la infraestructura subyacente, la seguridad, el mantenimiento y las actualizaciones”.

Algunos ejemplos son: Google Workspace, Microsoft 365, Dropbox.

2.6.2 Amazon Web Services (AWS)

Amazon Web Services (AWS), fue creada en 2006 por Amazon, esta proporciona más de 200 servicios en múltiples áreas, como lo son: la infraestructura, una plataforma y programas de computación en la nube. Fue desarrollada para asistir a empresas, programadores en el desarrollo y expansión de aplicaciones de forma segura y adaptable. Según Amazon (2024), "AWS es la plataforma en la nube más completa y ampliamente adoptada del mundo, que ofrece servicios con funcionalidades completas desde centros de datos a nivel global." En la Figura 24 ilustra de manera grafica los principales servicios de AWS con mayor demanda por los usuarios a lo largo de su desarrollo.

Figura 24

Representación de los principales servicios de AWS



Nota. Esta ilustración fue tomada de AllCode.

Los modelos de atención en la nube (IaaS, PaaS y SaaS) representan diferentes niveles de abstracción tecnológica, cada uno con ventajas y circunstancias de uso específicas. En la Tabla 8 muestra una comparativa técnica de sus principales diferencias en múltiples aspectos como la gestión, responsabilidad, con el fin de simplificar la elección del modelo adecuado en función de las necesidades técnicas y operativas.

Tabla 8

Comparación técnica entre múltiples proveedores de nube

Aspecto	IaaS	PaaS	SaaS
Nivel de control	Alto (configuración del SO, y el middleware).	Medio (aplicación)	Bajo (configuración de la aplicación)
Responsabilidad	Usuario gestiona SO y aplicación.	Proveedor gestiona infraestructura y SO	Proveedor gestiona todo
Escalabilidad	Manual	Automática	Automática
Costo	Pago por recursos	Pago por plataforma	Pago por suscripción
Mantenimiento	A cargo el usuario	Compartido	Gestionado por el proveedor
Tiempo de implementación	Largo	Medio	Bajo
Casos de uso	Infraestructura de servidores y backup	Desarrollo de aplicaciones y APIs	Correo, CRM y ERP.
Ejemplos	Amazon EC2, Azure VM	Google App Engine y Heroku	Gmail y Google Docs

2.6.2.1 Características generales

Las funcionalidades clave de Amazon Web Services incluyen:

- Escalabilidad: Es posible ajustar los recursos de acuerdo a la necesidad del usuario.
- Costos: Los usuarios únicamente pagan por lo que consumen.
- Accesibilidad: Se suministra una amplia infraestructura a nivel mundial distribuida en áreas de disponibilidad.
- Confiabilidad: Posee la conformidad con normas como ISO 27001 y GDPR.
- Beneficios: Ofrece un extenso espectro de servicios, que incluyen informática, almacenamiento, inteligencia artificial y bases de datos.

Estas características convierten a AWS en una de las plataformas de uso más frecuente en contextos empresariales, científicos y de estudio.

En la Figura 25 muestra de manera gráfica un mapa mundial con la distribución geográfica de las zonas de disponibilidad y regiones de Amazon Web Services (AWS), una de las plataformas de servicios en la nube más utilizadas a nivel global.

Figura 25

Red global de regiones de AWS



Nota. AWS cuenta con la infraestructura en la nube más amplia a nivel mundial. Esta ilustración fue tomada de AWS.

2.6.2.2 Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 también conocido como Cloud Elastic Compute, es un servicio que proporciona capacidad de computación escalable en la nube, este promueve el despliegue inmediato de servidores virtuales, su adaptación a sistemas operativos personalizados y la gestión completa del entorno de ejecución. Entre sus ventajas destacan la facilidad para aumentar recursos, automatizar tareas mediante scripts, y admitir variados tipos de instancias diseñadas para diferentes requerimientos como lo son: procesamiento, memoria, GPU y red (AWS, 2025).

2.6.2.3 Amazon S3 (Simple Storage Service)

De acuerdo con AWS (2025), Amazon Simple Storage Service (Amazon S3) “es un servicio de almacenamiento de objetos que ofrece escalabilidad, disponibilidad de datos, seguridad y rendimiento. Se pueden almacenar, administrar, analizar y proteger cualquier cantidad de datos para prácticamente cualquier caso de uso, como los lagos de datos, las aplicaciones nativas en la nube y las aplicaciones móviles. Gracias a las clases de almacenamiento rentables y a las características de administración fáciles de usar, es posible optimizar los costos, organizar y analizar los datos y configurar controles de acceso detallados para cumplir con requisitos empresariales y de conformidad específicos.”

2.6.3 Alternativas a AWS

A pesar de que Amazon Web Services (AWS) se destaca en el sector de servicios en la nube, hay otras plataformas frecuentemente utilizadas y competitivas que brindan soluciones parecidas, cada una con atributos únicos. Google Cloud Platform (GCP), Microsoft Azure y Oracle Cloud Infrastructure (OCI) son algunas de las opciones más destacadas.

2.6.3.1 Google Cloud Platform

Google Cloud Platform (GCP) es la plataforma en la nube desarrollada por Google, esta proporciona servicios de infraestructura, plataforma y programas para desarrollo, análisis de datos, inteligencia artificial. Se caracteriza por su habilidad

para manejar grandes volúmenes de información, recursos para el aprendizaje automático y su integración con otros servicios. De acuerdo con GCP (2025), “Google Cloud consiste en un conjunto de recursos físicos, como computadoras y unidades de disco duro, y recursos virtuales, como máquinas virtuales (VMs), que se encuentran en los centros de datos en todo el mundo”.

Entre sus servicios principales incluyen: Google Compute Engine, Google Cloud Storage y Big Query.

2.6.3.2 Microsoft Azure

Microsoft Azure es la plataforma en la nube desarrollada por Microsoft, esta cuenta con una amplia variedad de servicios como lo son: redes, almacenamiento y bases de datos. Sin embargo, está relacionada con productos empresariales de esta compañía como Microsoft 365, Active Directory y Windows Server. Microsoft define Azure (2025), “es una plataforma de nube abierta y flexible que permite construir, implementar y administrar aplicaciones rápidamente a través de una red global de centros de datos”.

2.6.3.3 Oracle Cloud

OCI es la infraestructura de nube corporativa de Oracle, tiene el objetivo de proporcionar soluciones eficientes en bases de datos, programas de negocios y cargas de trabajo críticas. Cuenta con flexibilidad, seguridad y capacidad de automatización, sobresaliendo por su solidez en las implementaciones de Oracle Database. De acuerdo con Oracle (2024), “OCI ofrece servicios de computación diseñados para ejecutar aplicaciones empresariales exigentes”.

En la Tabla 9 muestra las principales características de las plataformas de cómputo en la nube.

Tabla 9*Comparativa entre las principales plataformas de cómputo en la nube*

Característica	AWS	Azure	GCP	OCI
Empresa	Amazon	Microsoft	Google	Oracle
Lanzamiento	2006	2010	2008	2016
Servicios principales	EC2, S3, Lambda, RDS.	Virtual Machines, Azure SQL, App Services.	Compute Engine, Cloud Storage.	Oracle DB, Compute, Object Storage.
Fortalezas	Variedad de servicios, escalabilidad.	Integración con software empresarial.	Integración con software empresarial	Optimizado para bases de datos Oracle
Modelo de pago		Pago por uso		
Zona de disponibilidad	+100 zonas en +30 regiones	+60 regiones	+40 regiones	+45 regiones
Seguridad y cumplimiento	SOC 1/2/3, ISO 27001, GDPR, FedRAMP	ISO, SOC, HIPAA, GDPR	ISO, SOC, FedRAMP	ISO, SOC, HIPAA, PCI

CAPÍTULO III. MARCO METODOLÓGICO

Este trabajo propone una metodología híbrida, ya que el proyecto integra dos áreas tecnológicas: desarrollo móvil e inteligencia artificial. El objetivo es aplicar las principales fases de cada área para mejorar la gestión y el desarrollo de este proyecto. Para el desarrollo móvil se adoptó la metodología ágil “Mobile-D”, mientras que para la inteligencia artificial se empleó la metodología “CRISP-DM”.

3.1 Mobile D

Es una metodología ágil diseñada para agilizar el desarrollo de software para dispositivos móvil, pensada para equipos de trabajo reducidos. Está basada en diferentes técnicas de otras metodologías como lo son: Extreme Programming, Rational Unified Process y Crystal Methodologies. Empleadas en prácticas de desarrollo, métodos de escalabilidad y el ciclo de vida de un proyecto (Rodríguez, A., 2019)

En la Figura 26 ilustra el ciclo de vida de Mobile-d, el cual consta de las siguientes fases: exploración, inicialización, producción, estabilización y pruebas.

Figura 26

Fases de la metodología Mobile-D.



Nota. Esta ilustración fue elaborada por Napkin.

A continuación, se describe cada una de las fases:

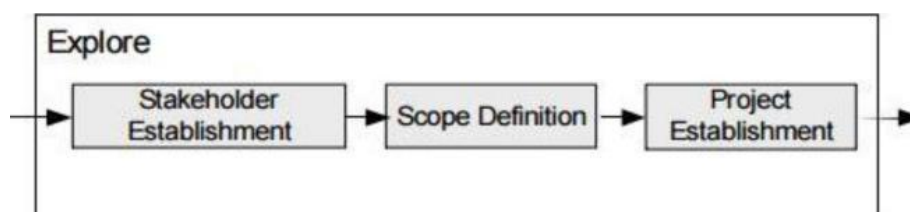
3.1.1 Fase de exploración

En esta etapa se establece una visión general del proyecto para el desarrollo futuro, se definen los requisitos y alcances, así como se establece la arquitectura que dará inicio al producto. Es una fase importante debido a que sienta las bases para la implementación controlada de software (Maraza J. & Rojas, M., 2021).

En la Figura 27 se presenta un diagrama general de los pasos a seguir de la fase mencionada.

Figura 27

Proceso en la fase de exploración.



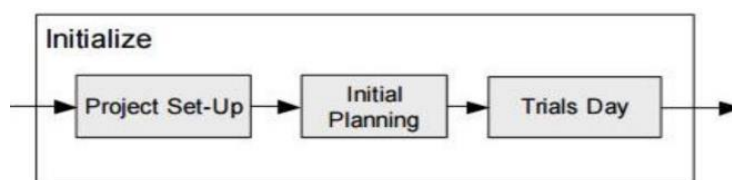
Nota. Esta ilustración fue tomada de *Mobile-D* (p. 19), por Koskela, 2004.

3.1.2 Fase de inicialización

En esta etapa tiene el objetivo de permitir el éxito en las siguientes etapas del proyecto mediante la preparación y verificación de problemas, en esta se definen los recursos físicos, tecnológicos y el equipo de trabajo (Alayo V. & Pinedo T., 2022). En la Figura 28 se presenta un diagrama general de los pasos a seguir de la fase mencionada.

Figura 28

Proceso en la fase de inicialización.



Nota. Esta ilustración fue tomada de *Mobile-D* (p. 21), por Koskela, 2004.

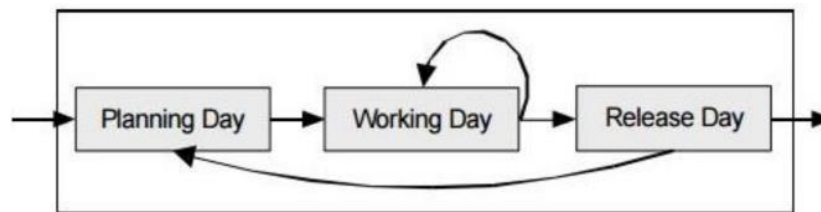
3.1.3 Fase de producción

De acuerdo con Rodríguez (2019), “El propósito de esta fase es implementar las funcionalidades que requiere el producto mediante el desarrollo iterativo e incremental.”

Esta se puede dividir en tres etapas: planificación donde se ejecutan las pruebas de aceptación, el día de trabajo donde se desarrollan las funciones seleccionas y día de entrega donde se entrega la funcionalidad desarrollada, como lo muestra de manera grafica la Figura 29.

Figura 29

Proceso en la fase de producción.



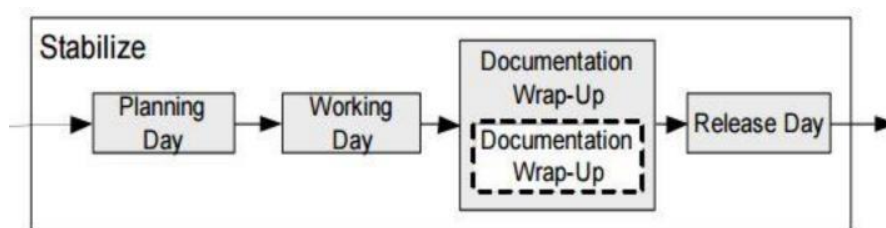
Nota. Esta ilustración fue tomada de *Mobile-D* (p. 23), por Koskela, 2004.

3.1.4 Fase de estabilización

En esta fase se realiza la integración de los módulos individuales en una sola aplicación para asegurar que el sistema funcione correctamente. En la Figura 30 presenta un diagrama general donde muestra los objetivos de esta fase siendo ellos el mejorar y garantizar la calidad del producto, así como realizar la documentación del proyecto (Sepa F., 2022).

Figura 30

Proceso en la fase de estabilización.



Nota. Esta ilustración fue tomada de *Mobile-D* (p. 24), por Koskela, 2004.

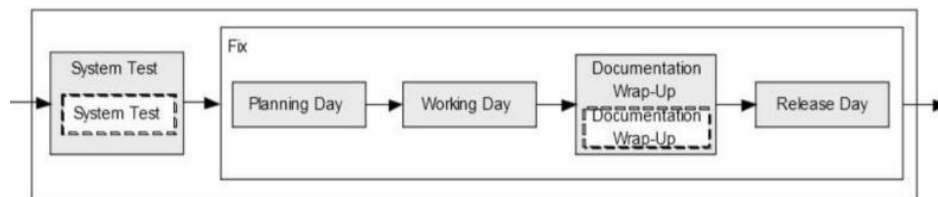
3.1.5 Fase de pruebas

El objetivo final de esta fase es que la aplicación móvil sea funcional y estable para el uso de los clientes, una vez el desarrollo de la aplicación haya concluido se integra y realiza distintas pruebas basándose en los requerimientos del cliente, así como el posible manejo de errores posibles descubiertos durante esta fase. (Alayo V. & Pinedo T., 2022).

En la Figura 31 presenta un diagrama general de los pasos a seguir de la fase mencionada.

Figura 31

Proceso en la fase de pruebas.



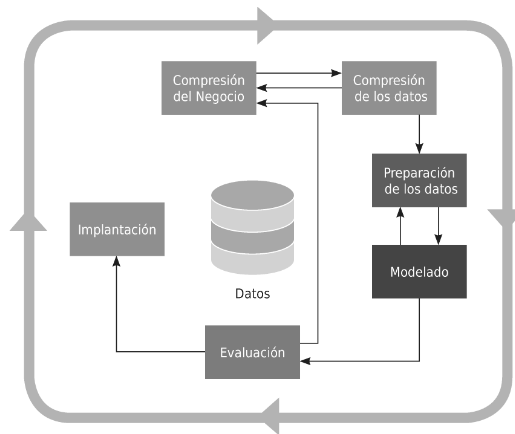
Nota. Esta ilustración fue tomada de *Mobile-D* (p. 26), por Koskela, 2004.

3.2 CRISP-DM

De acuerdo con Castillo J. (2019), “la metodología CRISP-DM establece las etapas necesarias para la realización de proyectos de minería de datos. Es una guía de acceso gratuito basada en el proceso KDD”. En la Figura 32 ilustra las seis fases principales: comprensión del negocio, comprensión de los datos, preparación de datos, modelado, evaluación e implementación.

Figura 32

Diagrama de la metodología CRISP-DM.



Nota. Esta ilustración fue tomada de *Mobile-D* (p. 26), por Koskela, 2004

Fuente: Castillo Romero, J. A. (2019). Big data. IFCT128PO. IC Editorial.

3.2.1 Compresión del problema

Esta es la etapa más importante de esta metodología debido a que permite entender el problema que posteriormente definirá las siguientes etapas a seguir. Las principales actividades de esta etapa son:

- Identificación del problema: Entender y delimitar la problemática, así como identificar los requisitos, supuestos, restricciones y beneficios del proyecto.
- Determinación de objetivos: Puntualizar las metas a lograr al proponer una solución (Zúñiga, E. ,2020).

3.2.2 Compresión de datos

En esta etapa se obtienen los datos a utilizar en el proyecto identificando las fuentes, las técnicas empleadas para su recolección, los problemas encontrados para su obtención y la forma en la que se resolvieron.

Además, se exploran a detalle estos datos con el fin de familiarizarse con ellos, al examinar las características clave, la calidad y la estructura de datos a través de técnicas como el análisis de datos y la visualización (Santana, F., 2023).

3.2.3 Preparación de datos

En esta etapa consiste en preparar el conjunto de datos para posteriormente realizar la aplicación de distintas técnicas de DM, generalmente es la etapa que consume más tiempo. Las principales actividades de esta etapa son:

- Limpieza de datos.
- Transformación de datos.
- Selección de tributos.
- Balanceo de datos (Sifuentes et al., 2020).

3.2.4 Modelado

En esta etapa se obtiene propiamente el modelo a implementar para el desarrollo del proyecto, las principales actividades:

- Selección de técnica de modelo: Se examina la técnica apropiada de acuerdo con el problema a resolver mediante los datos obtenidos, las herramientas y el dominio de la técnica escogida.
- Selección de datos de pruebas: Se requiere dividir la muestra en datos de entrenamiento y de validación.
- Obtención del modelo: Generar el modelo mediante un proceso iterativo de modificación de parámetros del mismo. (Zúñiga, E. ,2020)

3.2.5 Evaluación del modelo

En esta etapa se analizan los modelos de clasificación descubiertos en la etapa anterior, mediante las métricas estadísticas del mismo comparado con resultados previos o bien analizando los resultados con apoyo de expertos en el dominio del tema. Se realizan pruebas para asegurar que el modelo pueda generalizar bien en datos no vistos y que cumple con los requisitos (Sifuentes et al., 2020).

3.2.6 Implementación del modelo

Los modelos desarrollados se implementan en el entorno de producción y se monitorean para asegurar su rendimiento continuo, los resultados son presentados y los hallazgos pueden utilizarse para mejorar los procesos del negocio. Además,

se documentan los resultados de manera clara para que el usuario final, asimismo monitorear acciones para detectar áreas de oportunidad. (Santana, F., 2023).

CAPÍTULO IV. RESULTADOS

4.1 Compresión del problema

En el área de la microbiología un procedimiento fundamental es el conteo de las Unidades Formadoras de Colonias con el objetivo de establecer la cantidad de microorganismos presentes en una muestra, No obstante, este proceso se lleva a cabo de manera manual, este requiere que el análisis dure mucho tiempo, dependa de la experiencia del operador y pueda llegar a tener errores humanos, sobre todo cuando se procesan grandes cantidades de muestras o las colonias tienen condiciones variables de iluminación o se superponen.

En esta circunstancia, se hace necesario automatizar el procedimiento de conteo con el fin de disminuir el tiempo requerido para el análisis y mejorar la confiabilidad de los resultados. Mediante la visión artificial brinda las herramientas y técnicas para la identificación y cuantificación de objetos a través de imágenes.

El objetivo del proyecto es desarrollar un sistema basado en visión artificial que emplee un modelo YOLO para identificar y contar de manera automática las unidades formadoras de colonias que aparecen en fotografías de placas Petri.

Los objetivos son los siguientes:

- Reducir el tiempo que requieren los análisis microbiológicos mediante la automatización del procedimiento de conteo de colonias bacterianas.
- Apoyar en la toma de decisiones en la investigación científica o control de calidad.
- Disminuir la variabilidad sujeta a la interpretación del operado con el fin de estandarizar los resultados.
- Implementar el modelo desarrollado en una aplicación móvil.

El alcance del proyecto es de un modelo YOLO entrenado con imágenes de placas Petri, su validación frente a conteos manuales y la integración del modelo en una

aplicación móvil que permita realizar el conteo de forma automática a partir de imágenes capturadas o cargadas por el usuario.

4.2 Compresión de los datos

Para el desarrollo del modelo de detección y conteo de unidades formadoras de colonias, se realizó la búsqueda de los conjuntos de imágenes públicas específicamente aquellas que presentaran cajas Petris con un medio de cultivo y presentaran un crecimiento visible de colonias. Esto con el fin de reunir un conjunto de datos diverso que incluyera diversas especies microbianas, condiciones de iluminación y cantidad de crecimiento para que el modelo pudiera generalizar independiente del escenario.

4.2.1 Recolección de datos

Se identificación y seleccionaron varios conjuntos de imágenes de repositorios públicos, investigaciones científicas y proyectos desarrollados por diversos institutos. Estas ofrecen material visual con licencias abiertas que permiten su utilización para fines de investigación.

En la Tabla 10 describe principales conjuntos de imágenes empleados para el desarrollo de este proyecto junto a la información necesaria.

Tabla 10

Conjunto de imágenes empleadas para esta investigación.

Instituto/empresa	Especies	Cantidad	Etiquetado
Centre of Biotechnology and Fine Chemistry	E.coli	1,350	NO
	S.aureus		
	P.aeruginosa		
NeuroSYS Research	E.coli	.18,000	SI
	S.aureus		
	P.aeruginosa		
	B.subtilis		

	C.albicans		
Kaggle	E.coli	200	NO
MILAB Artificial Intelligence National Laboratory.	E.coli		
	S.aureus	75	SI
	P.aeruginosa		
TOTAL		19,625	

Se puede decir que el conjunto final de datos forma una base diversa para el entrenamiento del modelo de YOLO, debido a que presente variaciones en forma, color, tamaño en las instancias de las colonias.

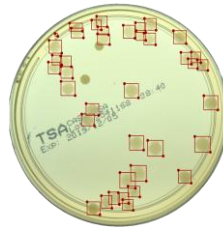
4.2.2 Especies microbianas a considerar

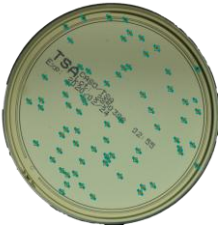
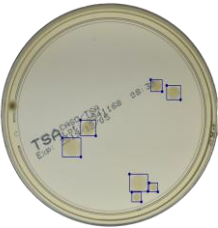
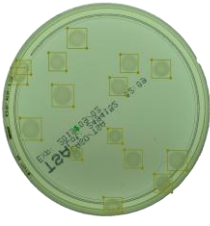
Dado al conjunto de imágenes recolectado se seleccionaron aquellas muestras que presentaron interés microbiológico general. Estas especies fueron elegidas debido a su relevancia científica, así como por su relevancia en investigaciones microbiológicas y su diversidad morfológica para evaluar la capacidad del modelo de detección en distintos tipos de colonias.

En la Tabla 11 presenta las especies microbianas incluidas en el desarrollo de este proyecto, así como una breve descripción y su relevancia.

Tabla 11

Especies microbianas consideradas en la detección automática de colonias.

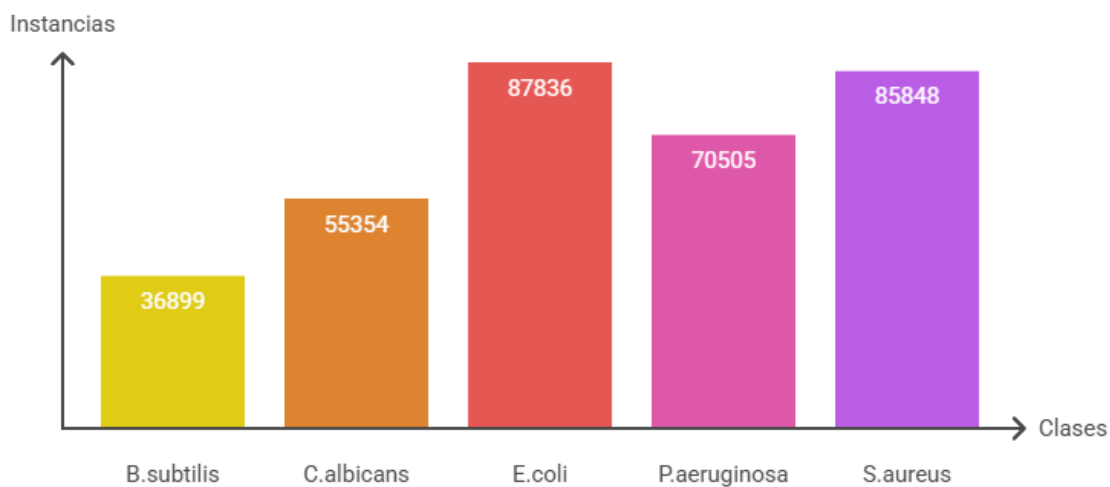
Nombre	Descripción	Morfología	Imagen
Escherichia coli	Es una bacteria Gram negativa, es utilizada como organismo modelo, es una indicadora de contaminación fecal y ampliamente presente en estudios de control sanitario.	Circulares, lisas, de color crema o blanquecino; crecimiento uniforme.	

Staphylococcus aureus	Es una bacteria Gram positiva patógena, está asociada a infecciones cutáneas y contaminaciones alimentarias.	Redondas, convexas, brillantes, con pigmento dorado característico	
Pseudomonas aeruginosa	Es una bacteria Gram negativa oportunista, frecuente en ambientes hospitalarios y muestras ambientales.	Irregulares, pigmentadas de tono verdoso o azulado, con bordes difusos.	
Bacillus subtilis	Es una bacteria Gram positiva formadora de esporas, ampliamente usada como modelo de laboratorio.	Grandes, opacas y rugosas; presentan textura seca y apariencia mate.	
Candida albicans	Es una levadura patógena de importancia médica, causante de infecciones.	Cremosas, lisas, de color blanco o marfil; superficie brillante y húmeda.	

La Figura 33 muestra la distribución del número de instancias por clase en el conjunto de datos utilizado para entrenar el modelo YOLO. Donde se observa que la clase E. coli cuenta con el mayor número de instancias (87,836), seguida de S. aureus (85,848) y P. aeruginosa (70,505), las clases con menor cantidad de muestras son C. albicans (55,354) y B. subtilis (36,899).

Figura 33

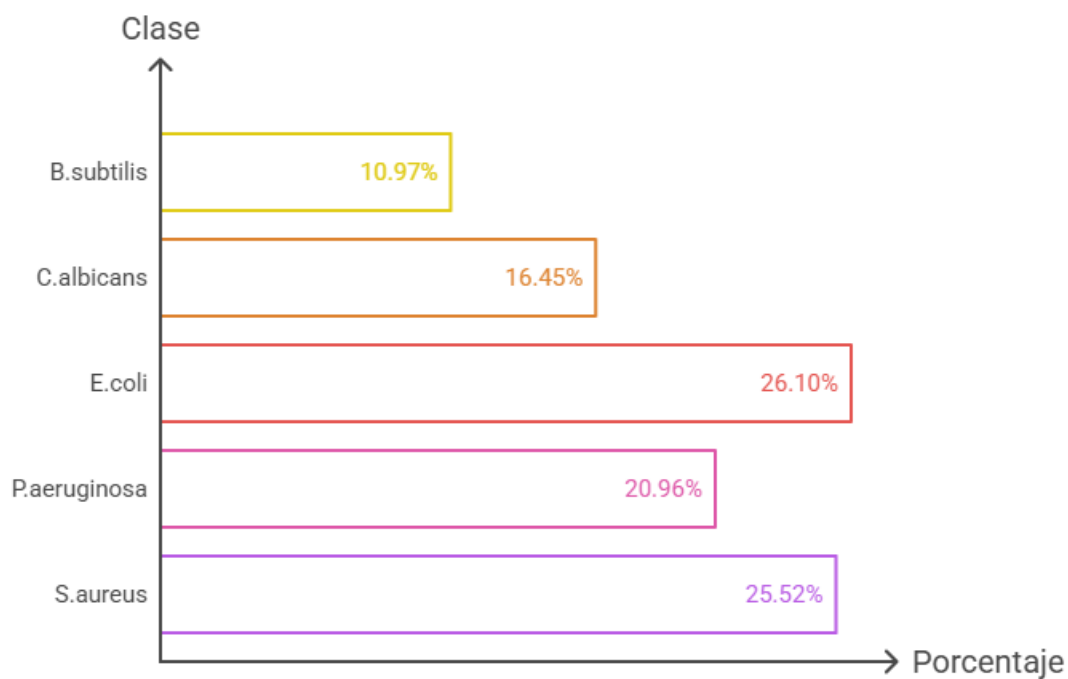
Distribución de instancias por clase.



En **Figura 34** ilustra gráficamente de manera porcentual la distribución de instancias por clase en el conjunto de datos.

Figura 34

Distribución porcentual por clase.



4.2.3 Clasificación de imágenes

El conjunto de imágenes recopiladas fue clasificado de acuerdo a lo siguiente:

- Condición de iluminación al momento de capturar la imagen.
 - Clara: Imagen capturada bajo iluminación intensa y fondo uniforme.
 - Intermedia: Imagen capturada con iluminación media.
 - Oscura: Imagen con condiciones de baja iluminación
 - Pobre: Imagen con distorsión lumínica.
- Tipo de muestra: Numero de colonias presentes en la muestra
 - Vacía: Placa sin crecimiento microbiano visible.
 - Contable: Placa con colonias bien definidas.
 - Incontable: Placa con gran presencia de colonias que impide la diferenciación individual.

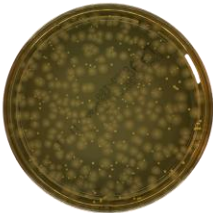
En la Tabla 12 detalla las categorías definidas anteriormente tomando el tipo de muestra según el número de colonias, la condición lumínica y su representación visual.

Tabla 12

Clasificación de imágenes.

Clasificación	Subclasificación	Cantidad	Imagen
Vacías	Claros	309	
	Oscuros	623	

	Intermedias	23	
	Pobres	262	
	Claras	994	
	Oscuras	5,998	
Contables			
	Intermedias	857	
	Pobres	4424	
Incontables	Claras	309	

Obscuras	623	
Intermedias	23	
Pobres	262	

4.3 Preparación de datos

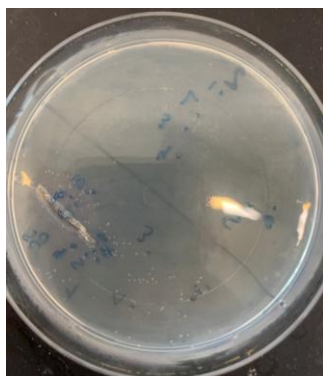
En esta fase se llevó a cabo el procesamiento y limpieza del conjunto de datos para asegurar que las imágenes empleadas para el entrenamiento del modelo de YOLO. El objetivo principal es obtener un conjunto de datos limpio, equilibrado que permitirá a la red neuronal identificar las clases microbianas bajo diferentes condiciones.

4.3.1 Selección de imágenes

Durante esta etapa se descartaron todas aquellas imágenes que estuvieran borrosas, que tuvieran reflejos, baja resolución o presencia de objetos ajenos a la caja Petri. En la Figura 35 ilustra de manera grafica un ejemplo de las muestras que no fueron consideradas para el entrenamiento del modelo.

Figura 35

Ejemplo de muestra eliminada.



4.3.2 Normalización y etiquetado

Las imágenes originales del conjunto de datos contaban con la resolución de 3640 píxeles de ancho por 4000 píxeles de alto, primero se redimensionaron todas las imágenes a una resolución 1024x1024 píxeles con el fin de optimizar el proceso de entrenamiento de YOLO. Esta normalización permite estandarizar el tamaño de la imagen de entrada a la red neuronal y simplifico el cálculo de las coordenadas de las instancias de las colonias en la imagen.

Se realizo el etiquetado de las colonias microbianas el cual es un paso fundamental para el entrenamiento del modelo. En la Figura 36 muestra la estructura original del etiquetado en formato COCO.

Figura 36

Etiquetado en formato COCO.

```
{
  "colonies_number": 121,
  "labels": [
    {
      "class": "C.albicans",
      "height": 45,
      "id": 1,
      "width": 45,
      "x": 285,
      "y": 2317
    },
    {
      "class": "C.albicans",
      "height": 45,
      "id": 2,
      "width": 45,
      "x": 341,
      "y": 2455
    }
  ]
}
```


Inicialmente las anotaciones se encontraban en formato COCO, ampliamente usando en proyecto de visión, pero no es compatible con la arquitectura de YOLO por lo que se realizó una conversión a formato TXT. Este formato incluye para cada objeto la clase a la que pertenece y las coordenadas normalizadas del cuadro delimitador (bounding box) que contiene la colonia. En la Figura 37 se muestra el resultado de la conversión del etiquetado de formato a COCO a YOLO.

Figura 37

Etiquetado en formato YOLO.

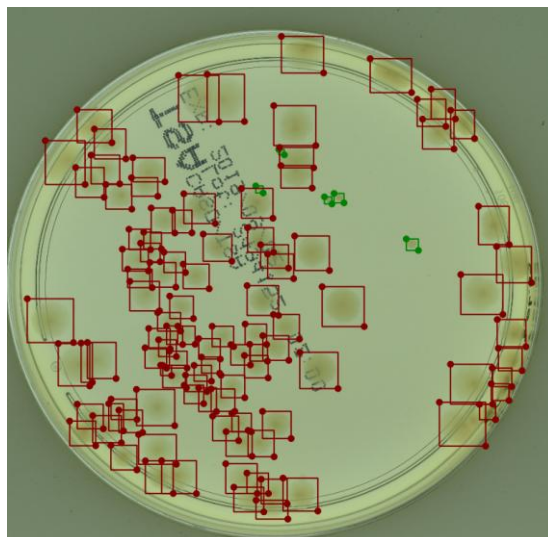
```
0 0.089027 0.584875 0.013028 0.011250
0 0.105240 0.619375 0.013028 0.011250
0 0.115373 0.624625 0.013028 0.011250
0 0.122322 0.653625 0.013028 0.011250
0 0.160249 0.622375 0.013028 0.011250
0 0.245368 0.618625 0.013028 0.011250
0 0.109004 0.558375 0.013028 0.011250
0 0.098292 0.542625 0.013028 0.011250
0 0.121743 0.550375 0.013028 0.011250
0 0.121743 0.543875 0.013028 0.011250
0 0.106398 0.427625 0.013028 0.011250
0 0.118848 0.436125 0.013028 0.011250
0 0.189490 0.391125 0.013028 0.011250
0 0.312826 0.414625 0.013028 0.011250
0 0.345831 0.403375 0.013028 0.011250
0 0.330776 0.364125 0.013028 0.011250
0 0.340909 0.370375 0.013028 0.011250
0 0.389259 0.373625 0.013028 0.011250
```

Una vez realizado el etiquetado del conjunto de imágenes, es necesario ser visualizadas y verificadas esas etiquetas mediante herramientas como lo son Labelme. El objetivo es verificar la precisión y consistencia de los cuadros delimitadores antes de comenzar el entrenamiento del modelo.

En la Figura 38 se puede visualizar una imagen etiquetada en la herramienta LabelMe, donde se aprecian los cuadros delimitadores que indican la ubicación de cada colonia microbiana.

Figura 38

Visualización de imagen etiquetada.



Cada objeto etiquetado fue asociado a una de las clases definidas:

- Escherichia coli
- Staphylococcus aureus
- Pseudomonas aeruginosa
- Bacillus subtilis
- Candida albicans

4.4 Modelado

Se utilizaron múltiples herramientas de hardware y software para llevar a cabo el desarrollo y entrenamiento del modelo de detección de unidades formadoras de colonias, los elementos empleados se describen a continuación:

- Ultralytics (versión 8.3): Es un framework que permite la implementación de modelos de YOLO.
- Python (versión 3.13.5): Es un lenguaje de programación utilizado para el proyecto.
- PyTorch (versión 2.8.0): Es una biblioteca que permite el entrenamiento y despliegue de modelos de aprendizaje profundo.

- CUDA (versión 12.8): Es una plataforma para el cómputo paralelo de NVIDIA que permite la aceleración mediante la GPU.
- YOLO (versión 11n y 12n): Es un modelo de detección de objetos y segmentación de imágenes mediante aprendizaje profundo.
- Tarjeta de video NVIDIA RTX 3060 (12 GB VRAM): Es el hardware empleado para el entrenamiento del modelo.

En el entrenamiento del modelo de detección se realizaron diversas configuraciones estructurales que influyen directamente en el desempeño, estabilidad y velocidad para llevar a cabo el aprendizaje. En la Tabla 13 muestra los principales parámetros utilizados durante el desarrollo del modelo, así como una breve descripción y el valor asignado para cada uno de ellos.

Tabla 13

Hiperparámetros utilizados para el entrenamiento del modelo.

Parámetros	Descripción	Valor
Tamaño del lote (batch)	Es la cantidad de imágenes procesadas de manera simultánea.	8
Número de Épocas (epochs)	Es el pase completo hacia adelante y hacia atrás de todas las imágenes de entrenamiento.	400
Tamaño de la imagen (imgsz)	Es el tamaño de imagen de entrada al entrenamiento.	1,024 x 1,024
Paciencia (patience)	Es el número de épocas que se esperan sin mejora antes de detener el entrenamiento anticipadamente.	25
Tasa de aprendizaje (lr0)	Determina el tamaño en cada iteración mientras se avanza hacia un mínimo en la función de pérdida.	0.01

Factor de tasa de aprendizaje final (lrf)	Controla la cantidad que disminuye la tasa de aprendizaje durante el entrenamiento.	0.001
Optimizador (optimizer)	Se selecciona el optimizador para el entrenamiento.	SDG

Durante el entrenamiento del modelo se implementan diversas técnicas de aumento de datos con el propósito de incrementar el conjunto de imágenes aplicando una variabilidad y mejorar la capacidad de generalización del modelo. Es decir, estas técnicas permiten simular otros entornos y condiciones que podrían ser reales. En la Tabla 14 muestra los principales atributos utilizados junto a una breve descripción y el valor asignado para cada uno de ellos.

Tabla 14

Técnicas de aumentos de datos utilizados para el entrenamiento del modelo.

Atributo	Descripción	Valor
hsv_s	Modifica la saturación de una imagen afectando la intensidad de los colores.	0.0
hsv_v	Modifica el brillo de una imagen, afecta diversas condiciones de iluminación.	0.0
hsv_h	Modifica el tono de la imagen, introduciendo variabilidad de color.	0.0
flipud	Invierte la imagen verticalmente.	0.5
fliplr	Invierte la imagen de izquierda a derecha.	0.5
mosaic	Combina cuatro imágenes en una.	0.0
erasing	Elimina aleatoriamente regiones de la imagen.	0.0
scale	Escala la imagen.	0.0
copy_paste_mode	Especifica el copy-paste estrategia a utilizar.	'mixup'
translate	Traslada la imagen horizontal y verticalmente.	0.03

4.5 Evaluación del modelo

Se realizó el entrenamiento del modelo propuesto implementando dos versiones distintas de YOLO. Las seleccionadas fueron las siguientes versiones YOLOv11 y YOLOv12. A continuación se describen las características principales de cada una:

- YOLOv11: Mejora la eficiencia, la rapidez y optimiza la arquitectura del backbone mediante bloques de atención, además de permitir un rendimiento más equilibrado entre la precisión y la velocidad. Su diseño permite la extracción de mapas de características con un tamaño de 256 canales en la capa P3, siendo un factor para la identificación de patrones finos y mejorar el rendimiento en los objetos de menor tamaño.
- YOLOv12: Es la versión más reciente del modelo, se destaca debido a la composición más profunda del backbone y tener una capacidad de representación superior. Los mapas de características de la capa P3 están formados por 128 canales debido a sus modificaciones en su arquitectura que buscaba minimizar el sobreajuste y optimizar la generalización.

En la Tabla 15 muestra los resultados obtenidos de la comparación cuantitativa entre ambas versiones de Yolo, considerando el tiempo de inferencia, el procesamiento y la complejidad computacional. Estos valores fueron obtenidos a partir de un análisis de 2,562 imágenes que fueron procesadas a través del modelo propuesto.

Tabla 15

Evaluación de modelos de Yolo

Tiempo (ms)	Yolo 11	Yolo12
Mínimo	133	229
Máximo	479	433
Promedio	186	279

Desviación estándar	0.0281	0.00237
Numero de parámetros	2,590,035	2,568,243
Numero de GLOPs	6.4	6.5

A partir de los resultados obtenidos, se puede determinar que YOLOv11 puede presentar un mejor rendimiento en cuanto tiempo de velocidad en la que realiza una inferencia dado a que presentan tiempos mínimos, máximos y promedios menores. Sin embargo, YOLOv12 demuestra ser mas estable durante el proceso de la inferencia dado a una menor desviación estándar y una menor cantidad de parámetros, es decir podría ser conveniente en escenarios que requieran mayor consistencia en tiempo de ejecución.

Para evaluar el desempeño de ambos modelos en términos de exactitud de detección se realizo una matriz de verdad por modelo considerando diferentes umbrales de Intersección sobre unión (IoU). Dado a este análisis permite identificar los verdaderos positivos (VP), falsos positivos (FP), falsos negativos (FN) y verdaderos negativos (VN) conforme a las inferencias generadas por el modelo y al etiquetado real de la imagen. Una vez obtenidos estos valores se calcularon métricas de evaluación como Precisión, Recall, F1-Score y Accuracy con el objetivo de determinar el umbral de IoU que proporciona un mejor equilibrio. En la Tabla 16 corresponde a los resultados obtenidos por cada umbral de IoU analizado en YOLOv11.

Tabla 16

Matriz de verdad obtenida de umbrales de IoU en YOLOv11.

IoU	VP	FP	FN	VN	Precisión	Recall	F1	Accuracy
0.1	56,438	813	9,647	237	0.8903	0.8267	0.8522	0.9091
0.2	60,416	911	5,669	237	0.8899	0.8535	0.8672	0.9353

0.3	61,518	968	4,567	237	0.8896	0.8624	0.8718	0.9438
0.4	61,700	1,007	4,385	237	0.8894	0.8640	0.8725	0.9451
0.5	61,738	1,033	4,347	237	0.8892	0.8642	0.8725	0.9452
0.6	61,773	1,075	4,312	237	0.8890	0.8644	0.8725	0.9452
0.7	61,794	1,320	4,291	237	0.8870	0.8645	0.8719	0.9429
0.8	61,819	1,563	4,266	237	0.8860	0.8646	0.8712	0.9429
0.9	61,847	3,000	4,238	237	0.8634	0.8649	0.8595	0.9226

En la Tabla 17 corresponde a los resultados obtenidos por cada umbral de IoU analizado en YOLOv12 con el fin de realizar el análisis comparativo de cada modelo propuesto.

Tabla 17

Matriz de verdad obtenida de umbrales de IoU en YOLOv12.

IoU	VP	FP	FN	VN	Precisión	Recall	F1	Accuracy
0.1	56,484	735	9,601	239	0.8912	0.8269	0.8530	0.9111
0.2	60,513	844	5,572	239	0.8907	0.8534	0.8678	0.9367
0.3	61,615	897	4,470	239	0.8905	0.8624	0.8725	0.9455
0.4	61,789	913	4,296	239	0.8904	0.8641	0.8733	0.9470
0.5	61,813	925	4,272	239	0.8903	0.8642	0.8733	0.9471
0.6	61,838	948	4,247	239	0.8902	0.8643	0.8733	0.9471
0.7	61,850	976	4,235	239	0.8900	0.8644	0.8733	0.9470
0.8	61,874	1,140	4,211	239	0.8893	0.8645	0.8730	0.9464
0.9	61,898	2,982	4,187	239	0.8781	0.8647	0.8674	0.9366

Se realizó un análisis comparativo entre ambas matrices de verdad donde se puede concluir que YOLOv12 presenta más verdaderos positivos indicando que detecta mas objetos de manera correcta además de contener menos falsos negativos. Se obtuvo que YOLOv12 es 0.1% más preciso y su F1-Score es 0.008% mayor. Además, se determinó que su IoU mas estable es el valor 0.5 - 0.6, por ende, se

puede llegar a la conclusión que YOLOv12 tiene un mejor rendimiento en cuanto a la detección de objetos.

Para complementar el análisis comparativo se generó una matriz de verdad modificando el umbral de confianza del modelo, con el fin de evaluar la detección donde el modelo puede llegar a ser más estricto o permisivo en las inferencias realizadas. En la Tabla 18 muestra los resultados obtenidos por cada umbral de confianza en el modelo de YOLOv11.

Tabla 18

Matriz de verdad obtenida de umbrales de confianza en YOLOv11.

Conf	VP	FP	FN	VN	Precisión	Recall	F1	Accuracy
0.1	63961	6894	2124	233	0.8523	0.8877	0.8665	0.9288
0.2	63535	3952	2550	235	0.8726	0.8826	0.8754	0.9445
0.3	63091	2542	2994	236	0.8809	0.8770	0.8765	0.9483
0.4	62563	1703	3522	237	0.8851	0.8711	0.8718	0.9479
0.5	61716	1178	4369	237	0.8883	0.8636	0.8718	0.9439
0.6	59907	750	6178	239	0.8859	0.8457	0.8602	0.9298
0.7	55728	402	10357	240	0.8809	0.8080	0.8337	0.8955
0.8	39168	86	26917	241	0.8126	0.5998	0.6604	0.6906
0.9	57	0	66028	241	0.0171	0.0032	0.0046	0.0951

En la Tabla 19 muestra los resultados obtenidos por cada umbral de confianza en el modelo de YOLOv12 con el fin de realizar el análisis comparativo de cada modelo propuesto.

Tabla 19

Matriz de verdad obtenida de umbrales de confianza en YOLOv12.

Conf	VP	FP	FN	VN	Precisión	Recall	F1	Accuracy
------	----	----	----	----	-----------	--------	----	----------

0.1	63806	4570	2279	234	0.8704	0.8847	0.8753	0.9439
0.2	63398	2691	2687	235	0.8813	0.8796	0.8783	0.9503
0.3	63004	1805	3081	237	0.8851	0.8745	0.8774	0.9508
0.4	62531	1342	3554	238	0.8878	0.8701	0.8760	0.9496
0.5	61764	977	4321	239	0.8900	0.8635	0.8728	0.9461
0.6	60344	672	5741	239	0.8882	0.8507	0.8643	0.9352
0.7	57016	395	9069	239	0.8835	0.8209	0.8432	0.9077
0.8	45044	97	21041	240	0.8560	0.6927	0.7442	0.7828
0.9	737	0	65348	241	0.8560	0.0255	0.0336	0.1174

Dado a esta comparación al variar los umbrales de confianza en ambos modelos, demuestra que YOLOv12 proporciona un comportamiento mas estable y equilibrado, dado a que los umbrales bajos obtienen una relación favorable entre verdaderos positivos (VP) y falsos positivos (FP) que podría significar una mejor precisión. Además de obtener valores mas altos y constantes en cuanto al F1 score además de obtener el umbral de confianza de 0.3.

4.6 Implementación del modelo

CAPÍTULO V. CÓMPUTO EN LA NUBE

Con la finalidad de atender la LGAC de cómputo en la nube se realiza la configuración y despliegue de una solución tecnológica realizando una o más pruebas escalables en la capa gratuita de AWS, utilizando el servicio EC2.

5.1 Creación de la estancia

Se llevó a cabo la implementación de un servidor en la nube que será el encargado de procesar las distintas solicitudes de la aplicación móvil. En la Figura 39 muestra la configuración de una instancia EC2 (Elastic Compute Cloud) de Amazon Web Services, donde se seleccionó el sistema operativo de Ubuntu Server en la versión 24.04 LTS.

Figura 39

Creación de instancia de EC2

EC2 > Instancias > Lanzar una instancia

Nombre y etiquetas Información

Nombre
EasyCountCol-SERVER [Agregar etiquetas adicionales](#)

▼ **Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon)** Información

Una AMI es una plantilla que contiene la configuración de software (sistema operativo, servidor de aplicaciones y aplicaciones) necesaria para lanzar la instancia. Busque o examine las AMI si no ve lo que busca a continuación.

🔍 Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Recientes **Inicio rápido**

Amazon Linux macOS **Ubuntu** Windows Red Hat SUSE Linux

aws Mac ubuntu Microsoft Red Hat SUSE

[Buscar más AMI](#)
Inclusión de AMI de AWS, Marketplace y la comunidad

Imágenes de máquina de Amazon (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type [Apto para la capa gratuita](#)
ami-04f167a56786e4b09 (64 bits (x86)) / ami-0ae6f07ad3a8ef182 (64 bits (Arm))
Virtualización: hvm Activado para ENA: true Tipo de dispositivo raíz: ebs

Descripción
Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).
Canonical, Ubuntu, 24.04, amd64 noble image

Arquitectura ID de AMI Fecha de publicación Nombre de usuario [Proveedor verificado](#)

64 bits... ami-04f167a56786e4b09 2025-03-05 ubuntu

En la Figura 40 se selecciona el tipo de instancia que se empleara en el servidor de Ubuntu, siendo “t2.micro” debido a que forma parte de la capa gratuita que ofrece AWS.

Figura 40

Tipo de instancia

▼ Tipo de instancia [Información](#) | [Obtener asesoramiento](#)

Tipo de instancia

t2.micro Apto para la capa gratuita

Familia: t2 1 vCPU 1 GiB Memoria Generación actual: true

Bajo demanda Ubuntu Pro base precios: 0.0134 USD por hora

Bajo demanda Linux base precios: 0.0116 USD por hora

Bajo demanda SUSE base precios: 0.0116 USD por hora

Bajo demanda Windows base precios: 0.0162 USD por hora

Bajo demanda RHEL base precios: 0.026 USD por hora

☐ Todas las generaciones

[Comparar tipos de instancias](#)

Se aplican costos adicionales a las AMI con software preinstalado

▼ Par de claves (inicio de sesión) [Información](#)

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - obligatorio

Desarrollo ▼

[Crear un nuevo par de claves](#)

▼ Configuraciones de red [Información](#)

[Editar](#)

Red | [Información](#)

vpc-092c370094b53458c

Subred | [Información](#)

Sin preferencias (subred predeterminada en cualquier zona de disponibilidad)

Asignar automáticamente la IP pública | [Información](#)

Habilitar

Se aplican cargos adicionales cuando no se cumplen los límites del nivel gratuito

Firewall (grupos de seguridad) | [Información](#)

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☒ Crear grupo de seguridad

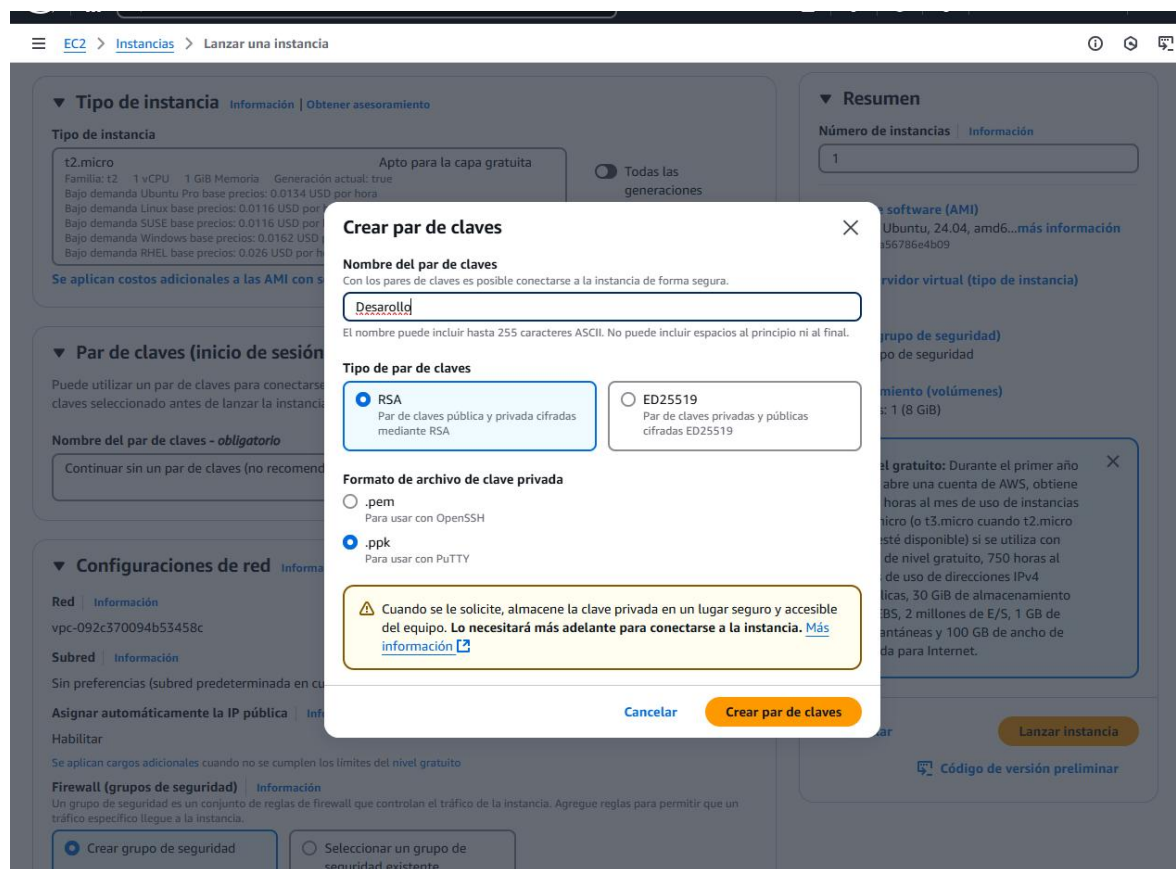
☐ Seleccionar un grupo de seguridad existente

107

Para permitir el acceso remoto al servidor se generaron las llaves de acceso necesarias para la conexión mediante el protocolo SSH, se realizó utilizando herramientas como PuTTY y PuTTYgen, para gestionar claves públicas y privadas en entornos Windows como puede apreciarse en la Figura 41.

Figura 41

Creación de llaves de acceso



Como se observa en la Figura 42, se realizó la configuración del grupo de seguridad asociado a la instancia donde se permite el tráfico a través del puerto 22 el cual es utilizado por el protocolo SSH. Esta regla de acceso fue establecida para aceptar conexiones desde cualquier dirección IP, para facilitar la administración remota desde distintas ubicaciones durante el proceso de desarrollo.

Figura 42

Definición de políticas de red

▼ Configuraciones de red [Información](#)

Editar

Red | [Información](#)

vpc-092c370094b53458c

Subred | [Información](#)

Sin preferencias (subred predeterminada en cualquier zona de disponibilidad)

Asignar automáticamente la IP pública | [Información](#)

Habilitar

Se aplican [cargos adicionales](#) cuando no se cumplen los límites del [nivel gratuito](#)

Firewall (grupos de seguridad) | [Información](#)

Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.

☒ Crear grupo de seguridad

☐ Seleccionar un grupo de seguridad existente

Crearemos un nuevo grupo de seguridad denominado "launch-wizard-2" con las siguientes reglas:

☒ Permitir el tráfico de SSH desde

Ayuda a establecer conexión con la instancia

Cualquier lugar
0.0.0.0/0

☐ Permitir el tráfico de HTTPS desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

☐ Permitir el tráfico de HTTP desde Internet

Para configurar un punto de enlace, por ejemplo, al crear un servidor web

⚠ Las reglas con origen 0.0.0.0/0 permiten que todas las direcciones IP tengan acceso a la instancia. Le recomendamos que configure las reglas del grupo de seguridad para permitir el acceso únicamente desde direcciones IP conocidas.

▼ Configurar almacenamiento [Información](#)

Avanzado

1x GiB

Adicionalmente, en la Figura 43 se asignó a la instancia un volumen de almacenamiento de 15 GB utilizando Amazon Elastic Block Store, debido a que se encuentra en el límite permitido por la capa gratuita de AWS, este espacio es suficiente para alojar el sistema operativo, las dependencias y los archivos del proyecto durante las fases iniciales del desarrollo.

Figura 43

Asignación de almacenamiento

The screenshot displays the AWS Management Console interface for launching an EC2 instance. At the top, there is a search bar and navigation breadcrumbs: **EC2** > **Instancias** > **Lanzar una instancia**. The main content area is divided into two sections:

- Configurar almacenamiento** (Configure storage): This section has tabs for **Información** (Information) and **Avanzado** (Advanced). It shows a configuration for 1x 15 GiB gp3 volumes. A blue information box states: "Los clientes que cumplan los requisitos de la capa gratuita pueden obtener hasta 30 GB de almacenamiento magnético o de uso general (SSD) de EBS". Below this is a button "Agregar un nuevo volumen" (Add new volume). A note mentions that the selected AMI contains more instance store volumes than allowed, and only the first 0 volumes are accessible. There is also a section for snapshots with a "Actualizar" (Update) button and a note about Data Lifecycle Manager policies. At the bottom of this section, it shows "0 x sistemas de archivos" (0 file systems) and an "Editar" (Edit) button.
- Detalles avanzados** (Advanced details): This section has tabs for **Información** (Information) and **Avanzado** (Advanced). It includes a "Directorio de unión al dominio" (Domain join directory) dropdown set to "Seleccionar", with a "Crear nuevo directorio" (Create new directory) link. Below that is a "Perfil de instancia de IAM" (IAM instance profile) dropdown set to "Seleccionar", with a "Crear un nuevo perfil de IAM" (Create new IAM profile) link. The "Tipo de nombre de anfitrión" (Host name type) section is partially visible at the bottom.

Una vez finalizada la configuración básica de la instancia donde se realizó la asignación de recursos, la generación de claves de acceso, la configuración del grupo de seguridad y del almacenamiento se procedió a lanzar la instancia EC2 como lo muestra en la Figura 44.

Figura 44

Lanzamiento de la instancia.

The screenshot shows the AWS Management Console interface for launching an EC2 instance. At the top, a green banner indicates that the launch was successful. Below this, a section titled 'Registro de lanzamiento' (Launch Record) lists the steps: 'Inicialización de solicitudes', 'Creación de grupos de seguridad', 'Creación de reglas de grupo de seguridad', and 'Inicio del lanzamiento', all marked as 'Se realizó correctamente'. The 'Pasos siguientes' (Next Steps) section includes three cards: 'Crear alertas de uso del nivel gratuito y facturación', 'Conectarse a la instancia', and 'Conectar una base de datos de RDS'. A button 'Ver todas las instancias' is located at the bottom right of the console area.

Como se observa en la Figura 45 se modificaron las reglas de entrada del grupo de seguridad para permitir el acceso a servicios específicos necesarios para el funcionamiento del sistema, para ello se habilitó el puerto 8000, utilizado por el servidor Uvicorn para ejecutar aplicaciones desarrolladas con FastAPI, y el puerto 3306 al servicio de base de datos MariaDB.

Figura 45

Permitir el tráfico en puertos específicos

The screenshot shows the 'Editar reglas de entrada' (Edit inbound rules) page in the AWS Management Console. The page title is 'Editar reglas de entrada' with a sub-link 'Información'. Below the title, a note states: 'Las reglas de entrada controlan el tráfico entrante que puede llegar a la instancia.' The main section is titled 'Reglas de entrada' and contains a table with the following columns: 'ID de la regla del grupo de seguridad', 'Tipo', 'Protocolo', 'Intervalo de puertos', 'Origen', and 'Descripción: opcional'. The table lists three rules: 1) SSH (Type: SSH, Protocol: TCP, Port Range: 22, Origin: Per...), 2) TCP personalizado (Type: TCP personalizado, Protocol: TCP, Port Range: 8000, Origin: An...), and 3) MySQL/Aurora (Type: MySQL/Aurora, Protocol: TCP, Port Range: 3306, Origin: An...). Each rule has an 'Eliminar' button. At the bottom left, there is an 'Agregar regla' button. A yellow warning banner at the bottom states: 'Las reglas cuyo origen es 0.0.0.0/0 o ::/0 permiten a todas las direcciones IP acceder a la instancia. Recomendamos configurar reglas de grupo de seguridad para permitir el acceso únicamente desde direcciones IP conocidas.' At the bottom right, there are three buttons: 'Cancelar', 'Previsualizar los cambios', and 'Guardar reglas'.

En la Figura 46 se estableció la conexión a la instancia EC2 mediante el protocolo SSH, con el objetivo de validar que el servidor estuviera funcionando correctamente.

Figura 46

Conexión a la instancia

```
welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1024-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Mon May  5 17:18:53 UTC 2025

System load:  0.46               Processes:            106
Usage of /:   12.4% of 13.49GB   Users logged in:     0
Memory usage: 20%               IPv4 address for enx0: 172.31.14.179
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-14-179:~$ sudo su
root@ip-172-31-14-179:/home/ubuntu#
```

La Figura 47 muestra la actualización del sistema operativo de la instancia EC2, así como la instalación de las paqueterías necesarias para el funcionamiento del proyecto.

Figura 47

Actualización e instalación de paquetes

```
ubuntu@ip-172-31-14-179:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [814 kB]
Get:8 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:9 http://us-east-2.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
```


En la Figura 48 se muestra la clonación del proyecto desde el repositorio alojado en GitHub, para obtener la versión más actualizada del código fuente del sistema.

Figura 48

Clonar el repositorio de GitHub del proyecto

```
ubuntu@ip-172-31-14-179:~$ git clone https://github.com/LuisVillal2/easycountcol
.git
Cloning into 'easycountcol'...
remote: Enumerating objects: 735, done.
remote: Counting objects: 100% (735/735), done.
remote: Compressing objects: 100% (457/457), done.
remote: Total 735 (delta 248), reused 692 (delta 205), pack-reused 0 (from 0)
Receiving objects: 100% (735/735), 9.15 MiB | 4.56 MiB/s, done.
Resolving deltas: 100% (248/248), done.
```

En la Figura 49 muestra la instalación de las librerías y dependencias necesarias para la ejecución del proyecto.

Figura 49

Instalación de los requerimientos de la aplicación

```
ubuntu@ip-172-31-14-179:~$ sudo apt install python3 python3-pip python3-venv mariadb-server -
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3 is already the newest version (3.12.3-0ubuntu2).
python3 set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 cpp
  cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot
  fontconfig-config fonts-dejavu-core fonts-dejavu-mono g++ g++-13
  g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu galera-4 gcc gcc-13 gcc-13-base
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu javascript-common
  libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl
  libaom3 libasan8 libatomic1 libbinutils libc-dev-bin libc-devtools libc6-dev
  libcc1-0 libcgi-fast-perl libcgi-pm-perl libclone-perl
  libconfig-inifiles-perl libcrypt-dev libctf-nobfd0 libctf0 libdbd-mysql-perl
  libdbi-perl libde265-0 libdeflate0 libdpkg-perl libencode-locale-perl
  libexpat1-dev libfakeroot libfcgi-bin libfcgi-perl libfcgi0t64
  libfile-fcntllock-perl libfontconfig1 libgcc-13-dev libgd3 libgomp1
  libgprofng0 libheif-plugin-aomdec libheif-plugin-aomenc
```

Como se observa en la Figura 50, se creó la base de datos en MariaDB en la instancia EC2 que será utilizada en el proyecto.

Figura 50

Creación de la base de datos

```
ubuntu@ip-172-31-14-179:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 10.11.11-MariaDB-0ubuntu0.24.04.2 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database easycountcol;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> 
```

En la Figura 51 muestra la restauración de las tablas correspondientes desde un archivo de respaldo con el comando “source”.

Figura 51

Ejecución de script para la restauración de la base de datos

```
ubuntu@ip-172-31-14-179:~/easycountcol$ sudo mysql -u luisvilla -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 32
Server version: 10.11.11-MariaDB-0ubuntu0.24.04.2 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use easycountcol;
Database changed
MariaDB [easycountcol]> source easycountcol.sql
Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

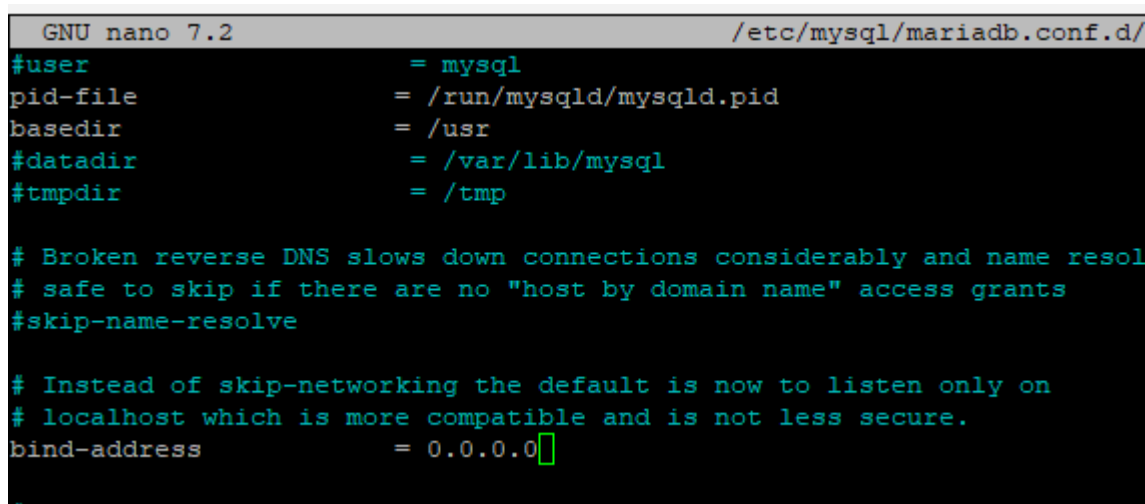
Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)
```

Se configuró MariaDB para permitir conexiones remotas desde cualquier ubicación, modificando los archivos de configuración correspondientes como puede apreciarse en la Figura 52.

Figura 52

Permitir conexiones remotas a la base de datos



```
GNU nano 7.2 /etc/mysql/mariadb.conf.d/
#user = mysql
pid-file = /run/mysqld/mysqld.pid
basedir = /usr
#datadir = /var/lib/mysql
#tmpdir = /tmp

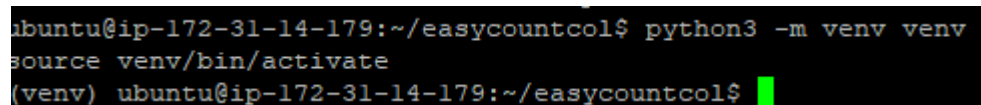
# Broken reverse DNS slows down connections considerably and name resol
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address = 0.0.0.0
```

En la Figura 53 se creó un entorno virtual en Python para aislar las dependencias del proyecto.

Figura 53

Creación de entorno virtual



```
ubuntu@ip-172-31-14-179:~/easycountcol$ python3 -m venv venv
source venv/bin/activate
(venv) ubuntu@ip-172-31-14-179:~/easycountcol$
```

Se realizó la ejecución del servicio de Uvicorn en la instancia de EC2 en segundo plano, siendo el encargado de gestionar las solicitudes HTTP para las aplicaciones desarrolladas con FastAPI como puede apreciarse en la Figura 54.

Figura 54

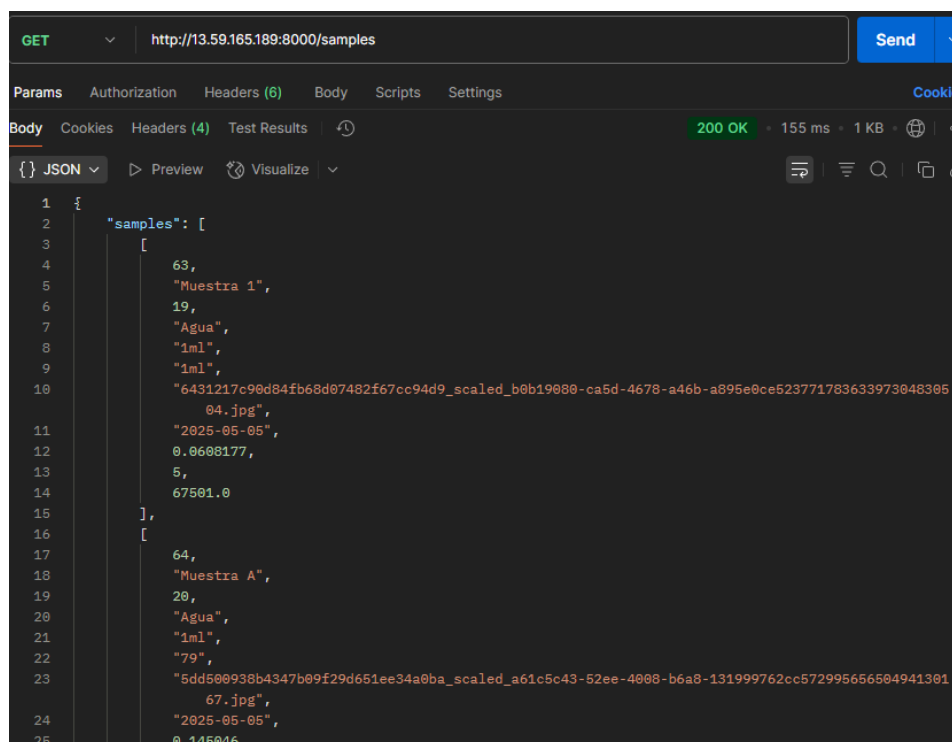
Ejecución de servidor de Uvicorn en EC2

```
ubuntu@ip-172-31-14-179:~/easycountcol$ cd lib/config/api/
ubuntu@ip-172-31-14-179:~/easycountcol/lib/config/api$ python3 -m venv venv
source venv/bin/activate
(venv) ubuntu@ip-172-31-14-179:~/easycountcol/lib/config/api$ nohup python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 > log.out 2>&1 &
[1] 29658
(venv) ubuntu@ip-172-31-14-179:~/easycountcol/lib/config/api$ ps aux | grep uvicorn
ubuntu 3872  0.1  9.2 340020 90916 ?        S1   May05  14:22 python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
ubuntu 29660 0.0  0.2   7076 2048 pts/0    S+   09:17   0:00 grep --color=auto uvicorn
(1)+ Exit 1      nohup python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 > log.out 2>&1
(venv) ubuntu@ip-172-31-14-179:~/easycountcol/lib/config/api$ ps aux | grep uvicorn
```

En la Figura 55 se realizó una prueba a uno de los endpoints de la aplicación FastAPI utilizando la IP pública asignada a la instancia EC2. Esta prueba permitió verificar que el servicio de Uvicorn estaba funcionando correctamente y que el servidor respondía adecuadamente a las solicitudes HTTP desde el exterior.

Figura 55

Prueba de endpoint



CAPÍTULO VI. CONCLUSIONES Y TRABAJO FUTURO

6.1 DISCUSIÓN

6.2 CONCLUSIONES

6.3 TRABAJO FUTURO

6.4 PUBLICACIONES

REFERENCIAS BIBLIOGRÁFICAS

Baron, S., Tenover, F. C., & Murray, P. R. (Eds.). (2019). *Microbiología médica* (4^a ed.). McGraw-Hill.

Food and Drug Administration (FDA). (2020). Bacteriological analytical manual (BAM): Aerobic plate count. <https://www.fda.gov/food/laboratory-methods-food/bam-aerobic-plate-count>

Synbiosis. (2024). *ProtoCOL 3 - Automatic colony counting and zone measuring equipment*. <https://www.synbiosis.com/product/automated-colony-counting-zone-measurement-protocol-3/Synbiosis>. (2024, September 17). *ProtoCOL 3 - Automatic colony counting and zone measuring equipment*. <https://www.synbiosis.com/product/automated-colony-counting-zone-measurement-protocol-3/>

BioLogics Inc. (s.f.). BioSpot colony counter. Recuperado el 5 de junio de 2025 de <https://www.biospot.com/>

Geissmann, Q. (2018). OpenCFU, a new free and open-source software to count cell colonies and other circular objects. *PLOS ONE*, 8(2), e54072. <https://doi.org/10.1371/journal.pone.0054072>.

Schneider, C. A., Rasband, W. S. y Eliceiri, K. W. (2012). NIH Image to ImageJ: 25 years of image analysis. *Nature Methods*, 9(7), 671–675. <https://doi.org/10.1038/nmeth.2089>

Prescott, L. M., Harley, J. P., & Klein, D. A. (2020). *Microbiology* (11th ed.). McGraw-Hill.

Zhang, Y., Jiang, H., Ye, T., & Juhas, M. (2020). Deep learning for imaging and detection of microorganisms. *Trends in Microbiology*, 29(7), 569-572. <https://doi.org/10.1016/j.tim.2020.11.002>

Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., y Zhang, X. (2018). End-to-end deep learning for self-driving cars: A comprehensive review. *IEEE Transactions on Intelligent Vehicles*, 3*(3), 247-258.

Christiansen, E. M., Yang, S. J., Ando, D. M., Javaherian, A., Skibinski, G., Lipnick, S. y Finkbeiner, S. (2021). Deep learning enables rapid identification of bacteria in microscopy images. *Nature Communications*, *12*(1), 1-13.

Litjens, G., Kooi, T., Bejnordi, B. E., Setio, A. A. A., Ciompi, F., Ghafoorian, M., y Sánchez, C. I. (2017). A survey on deep learning in medical image analysis. *Medical Image Analysis*, *42*, 60-88.

Ho, C. S., Jean, N., Hogan, C. A., Blackmon, L., Jeffrey, S. S., Holodniy, M. y Dionne, J. (2020). Raman spectroscopy and machine learning for bacterial identification. *Analytical Chemistry*, *92*(1), 436-445.

Bruno, E., Simões, M., & Tenreiro, R. (2019). Automated colony counting for high-throughput screening. *Journal of Microbiological Methods*, *157*, 24-29.

Pinto, F. R., Costa, R., & Lopes, J. D. (2021). Microbial quality control in food industry: The importance of colony counting methods. *Food Control*, 125, 107917. <https://doi.org/10.1016/j.foodcont.2021.107917>

Sutton, S. V. W. (2020). Environmental monitoring in pharmaceutical clean rooms: A risk-based approach. *PDA Journal of Pharmaceutical Science and Technology*, 74(1), 5-18. <https://doi.org/10.5731/pdaipst.2019.010843>

Esteva, A., et al. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), 115-118. <https://doi.org/10.1038/nature21056>

Liu, Y., et al. (2021). Computer vision for environmental monitoring: A review. *Science of The Total Environment*, 768, 144919. <https://doi.org/10.1016/j.scitotenv.2020.144919>

Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture*, 147, 70-90. <https://doi.org/10.1016/j.compag.2018.02.016>

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2018). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097-1105.

Schlegel, H. (1997). *Microbiología general*. Ediciones Omega.

U.S. Environmental Protection Agency EPA. (2020). Bioremediation principles. <https://clu-in.org/techfocus/default.focus/sec/Bioremediation/cat/Overview/>

Panel Intergubernamental sobre Cambio Climático IPCC. (2021). *Climate change 2021: The physical science basis. Contribution of Working Group I to the Sixth Assessment Report*. Cambridge University Press. <https://www.ipcc.ch/ar6-wg1/>

Grossart, H., Van Den Wyngaert, S., Kagami, M., Wurzbacher, C., Cunliffe, M., & Rojas-Jimenez, K. (2019). Fungi in aquatic ecosystems. *Nature Reviews Microbiology*, 17(6), 339-354. <https://doi.org/10.1038/s41579-019-0175-8>

Madigan, M. T., Bender, K. S., Buckley, D. H., Sattley, W. M., & Stahl, D. A. (2021). *Brock biology of microorganisms* (15th ed.). Pearson.

Calderón Peña, F. A., Burgos Villamil, M. A., & Castaño Henao, A. C. (2022). Ampliación del Laboratorio de Microbiología para la Industria Farmacéutica.

Stanier, R. Y., Ingraham, J. L., Wheelis, M. L., & Painter, P. R. (2024). *Microbiología*. Reverté.

Buchan, B. W., & Ledebor, N. A. (2022). *Advances in microbiological diagnostic technologies for bloodstream infections*. Clinical Microbiology Reviews, 35(3), e00079-21.

Contreras, J., Flores, T. G., Talavera, T. D. R. A., Martínez, Z. E., & López, N. A. P. (2017). ¿Qué son los microbios?, 224, 8-12.

Preciado, P. (2020). Aprendiendo del virus. *El país*, 28(3).

Mier, T., Toriello, C., & Ulloa, M. (2019). *Hongos microscópicos saprobios y parásitos*. UNAM.

International Organization for Standardization [ISO]. (2013). *ISO 7218:2013. Microbiology of food and animal feeding stuffs*.

Geovana, V. C. L. y Tatiana, V. F. (2017). *Clasificación de los microorganismos*. http://revistasbolivianas.ciencia.bo/scielo.php?script=sci_arttext&pid=S2304-37682014000500002&lng=pt&nrm=iso

Cabral, L. A., Pérez, M. J., & Gómez, S. C. (2020). Medios de cultivo en microbiología: fundamentos y aplicaciones. *Revista Ciencia y Tecnología para la Salud Visual y Ocular*, 18(1), 9-18. <https://doi.org/10.19052/sv.vol18.iss1.5>

Solmeglas (2023). Medios de cultivo: Qué son, Usos y Tipos - SOLMEGLAS - Solmeglas SL. Solmeglas SL. <https://solmeglas.com/medios-de-cultivo-que-son-funcionalidades-calidad/>

Comisión Federal para la Protección contra Riesgos Sanitarios COFEPRIS. (2023). *Catálogo de criterios y requisitos para el ingreso de muestras a análisis*. <https://www.gob.mx/cms/uploads/attachment/file/937158/CCAYAC-CT-07.pdf>

Rodríguez, J. M. (2018). *Microbiología aplicada: Técnicas de cultivo y diagnóstico* (2a ed.). Editorial Médica Panamericana. <https://www.medicapanamericana.com>

International Organization for Standardization [ISO]. (2013). *ISO 7218:2013. Microbiology of food and animal feeding stuffs*.

SAG (2022). Metodología de recuento de agentes microbianos de control de plagas y de contaminantes biológicos en productos microbianos de control de plagas. (pp. 1–27).

International Organization for Standardization [ISO]. (2024). *ISO 7218:2024. Microbiology of the food chain — General requirements and guidance for microbiological examinations*.

González, R. C., y Woods, R. E. (2018). *Digital image processing* (4th ed.). Pearson.

Botta, A., Cavallone, P., Baglieri, L., Colucci, G., Tagliavini, L., y Quaglia, G. (2022). A review of robots, perception, and tasks in precision agriculture. *Applied Mechanics*, 3(3), 830–854. <https://doi.org/10.3390/applmech3030049>

Peli, E. (2021). *Contrast in complex images: Perception and measurement*. *Journal of Vision*, 21(3), 1-23.

Igual, J. (2019). *El ruido en fotografía digital - foto igual*. Foto Igual. <https://fotoigual.com/tutoriales/el-ruido-en-fotografia-digital/>

Bojarski, M., Davide, D. T., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L. D., Monfort, M., Muller, U., Zhang, J., Zhang, X., Zhao, J., & Zieba, K. (2018). *End to End Learning for Self-Driving Cars*. arXiv.org. <https://arxiv.org/abs/1604.07316>

Tang, Y., Jean, N., Arévalo, P. y Friedl, M. A. (2024). Global forest monitoring with multi-sensor Earth observation and deep learning. *Remote Sensing of Environment*, 305, 114075.

Kumar, R., et al. (2023). *HSV-based preprocessing pipeline for AI-assisted endoscopic diagnosis*. *Medical Image Analysis*, 84, 102694.

Jensen, J. R. (2016). *Remote Sensing of the Environment: An Earth Resource Perspective* (2nd ed., p. 89). Pearson.

Phuangsaibai, N., Jakmunee, J., & Kittiwachana, S. (2021). Investigation into the predictive performance of colorimetric sensor strips using RGB, CMYK, HSV, and CIELAB coupled with various data preprocessing methods: a case study on an analysis of water quality parameters. *Journal of Analytical Science & Technology*, 12(1). <https://doi.org/10.1186/s40543-021-00271-9>

Miranda, M. (2018). *La imagen digital*. https://ve.scielo.org/scielo.php?script=sci_arttext&pid=S0016-35032009000200016

Thornbush, M. J. (2017). Measuring Surface Roughness through the Use of Digital Photography and Image Processing. *International Journal of Geosciences*, 05(05), 540–554. <https://doi.org/10.4236/ijg.2014.55050>

Querejeta, P. (2017). *Procesamiento digital de imágenes*. <http://lcr.uns.edu.ar/fvc/NotasDeAplicacion/FVC-QuerejetaSimbeniPedro.pdf>

Burger, W., & Burge, M. J. (2021). *Digital image processing: An algorithmic introduction* (3rd ed.). Springer. <https://doi.org/10.1007/978-3-030-72073-5>

Adobe Express. (2024). <https://www.adobe.com/es/express/discover/sizes/photo-aspect-ratio>

Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., Zhou, Y. (2023). TransUNet: Transformers make strong encoders for medical image segmentation. *Nature Communications*, 14(1), 3259.

Wang, Z., Zheng, L., Li, Y., & Wang, S. (2024). FaceXFormer: A unified transformer framework for facial analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(2), 1124-1136.

Cifuentes, Adriana (2019)."Desarrollo de una red neuronal convolucional para reconocer patrones en imágenes." *Investigación y desarrollo en TIC* 10.2: 7-17.

¿Qué es Computer Vision? | Microsoft Azure. (2025). <https://azure.microsoft.com/es-es/resources/cloud-computing-dictionary/what-is-computer-vision#clasificaci%C3%B3n-de-objetos>

Mahesh, B. (2020). Machine learning algorithms-a review. *International Journal of Science and Research (IJSR)*, 9(1), 381-386.

IBM. (2021). *Computer Vision*. Ibm.com. <https://www.ibm.com/mx-es/topics/computer-vision>

Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep learning*. MIT Press.

Wang, Z., et al. (2022). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(5), 1-20. <https://doi.org/10.1109/TPAMI.2022.3174035>

Amazon SageMaker AI. (2025). Métricas y validación, <https://docs.aws.amazon.com/es-es/sagemaker/latest/dg/autopilot-metrics-validation.html>

Scholnik-Elias, A., Martínez-Díaz, S., Luna-Taylor, J. E., & Castro-Liera, I. (2023). Detección de armas tipo pistola mediante el uso de redes convolucionales con una arquitectura tipo YOLO y estereoscopía. *Pädi Boletín Científico De Ciencias Básicas E Ingenierías Del ICBI*, 11(Especial2), 196–204. <https://doi.org/10.29057/icbi.v11iEspecial2.10727>

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.

Goodfellow, I., Courville, A., & Bengio, Y. (2023). *Deep learning* (2nd ed.). MIT Press. <https://www.deeplearningbook.org>

Calcagni, L (2020). *Redes Generativas Antagónicas y sus aplicaciones* (Doctoral dissertation, Universidad Nacional de La Plata).

Urtiaga, G (2020). *Administrar MySQL y MariaDB: Aprende a administrar MySQL y MariaDB fácilmente. AprendeIT*.

Dubois, P. (2021). *MySQL: The definitive guide to using, programming, and administering MySQL 8* (5th ed.). O'Reilly Media.

TensorFlow. (2025). *Computer vision with TensorFlow*. <https://www.tensorflow.org/tutorials/images>

Apple. (2023). *iOS Security Guide*. https://www.apple.com/business/docs/site/iOS_Security_Guide.pdf

Straub, B. y Chacon, S. (2021). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>

Rad, B. B., Bhatti, H. J., & Ahmadi, M. (2018). An introduction to docker and analysis of its performance. *International Journal of Computer Science and Network Security (IJCSNS)*, 17(3), 228.

Date, C. J. (2023). *An introduction to database systems* (9th ed.). Pearson.

Astigarraga, J (2022). ¿Se puede entender cómo funcionan Git y GitHub! *ecosistemas*, 31(1), 2332-2332.

Python (2024). Python Documentation. <https://docs.python.org/3/faq/general.html#what-is-python>

Pillow. (2025). *Pillow (PIL Fork) Documentation*. <https://pillow.readthedocs.io/>

NVIDIA (2025). *What is PyTorch?*. Data Science Glossary. <https://www.nvidia.com/en-us/glossary/pytorch/>

Pérez, R. M., Arias, J. S., & Porras, A. M. (2019). Introducción al aprendizaje automático con yolo. *Tecnología Vital*, 2(6).

Ramos, Dorys Lisette (2021). Aplicación de un modelo de reconocimiento de objetos utilizando Yolo (you only look once). La Libertad. UPSE, Matriz. Facultad de Sistemas y Telecomunicaciones. 47p

Rozada Raneros, S. (2021). Estudio de la arquitectura YOLO para la detección de objetos mediante deep learning.

OpenCV. (2025). *Open-Source Computer Vision Library*. <https://opencv.org/>

Morley, D., & Parker, C. S. (2021). *Understanding Computers: Today and Tomorrow* (17th ed.). Cengage Learning.

Microsoft. (2025). *Visual Studio Code*. <https://code.visualstudio.com/>

Melton, J., & Simon, A. R. (2002). *SQL:1999: Understanding relational language components*. Morgan Kaufmann.

Juárez, P., & Ivonne, I. (2018). Sistema gestor de base de datos. *REPOSITORIO REMERI*.

Google. (2025). *Build apps for any screen*. <https://flutter.dev>

OMS (2017). *Guidelines for Drinking-water Quality*.

Sadalage, P. J., & Fowler, M. (2024). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence* (2nd ed.). Addison-Wesley.

MongoDB Inc. (2024). *MongoDB Manual*. <https://www.mongodb.com/docs/manual/>

Cotrina-Teatino, M. A. ., Riquelme Sandoval, A. I. ., Guartan Medina, J. A. ., & Marquina Araujo, J. J. . (2025). Machine Learning aplicado a la exploración minera usando matriz de confusión. *SCIÉNDIO INGENIUM*, 21(1), 63-74. <https://doi.org/10.17268/rev.cyt.2025.01.06>

Borja-Robalino, R., Monleón-Getino, A., & Rodellar, J. (2020). Estandarización de métricas de rendimiento para clasificadores Machine y Deep Learning. *Revista Ibérica de Sistemas e Tecnologías de Informação*, (E30), 184-196.

Saji Santander, D. A. (2019). Reconocimiento de montos manuscritos en cheques a través de modelos de detección de objetos basados en redes convolucionales.

Liu, Fenghua & Zhao, Hongsheng & Liu, Weiguang. (2022). Improved garment detection algorithm based on YOLOv5. 1054-1058. 10.1109/WCMEIM56910.2022.10021514.

Fleuren, L. M., Klausch, T. L., Zwager, C. L., Schoonmade, L. J., Guo, T., Roggeveen, L. F., ... & Elbers, P. W. (2020). Machine learning for the prediction of sepsis: a systematic review and meta-analysis of diagnostic test accuracy. *Intensive care medicine*, 46(3), 383-400.

Introduction to Dart. (2025). <https://dart.dev/language>.

Microsoft Corporation. (2024). *SQL Server Documentation*. <https://learn.microsoft.com/en-us/sql/sql-server/>

Apple. (2023). *iOS Developer Documentation*. <https://developer.apple.com/documentation/ios>

Android Developers. (2025). *Android Studio*. <https://developer.android.com/studio>

Android Open-Source Project. (2023). *Platform Architecture*. <https://source.android.com/docs/core/architecture>

Uvicorn. (2024). *Uvicorn documentation*. <https://www.uvicorn.org/>

Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (Special Publication 800-145). U.S. Department of Commerce. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>

Google Cloud. (2024). *Google Cloud Platform*. <https://cloud.google.com/>

Richardson, L., & Ruby, S. (2021). *RESTful Web Services*. O'Reilly Media.

Ramírez, S. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com/>

Amazon Web Services. (2024). *Amazon EC2*. <https://aws.amazon.com/ec2/>

Oracle. (2024). *Oracle Cloud Infrastructure*. <https://www.oracle.com/cloud/>

Rittinghouse, J. W., & Ransome, J. F. (2017). *Cloud Computing: Implementation, Management, and Security*. CRC Press.

Amazon Web Services. (2024). *Amazon S3*. <https://aws.amazon.com/s3/>

Amazon AWS (2024). Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is-aws/>

Microsoft. (2024). *What is Azure?*. <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-azure/>

What is Software as a Service (SaaS)? | Microsoft Azure. (n.d.). <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-saas>

Beynon-Davies, P. (2018). *Sistemas de bases de datos*. Reverté.

Elmasri, R., & Navathe, S. B. (2018). *Fundamentals of database systems* (7th ed.). Pearson.

Rodríguez Suazo, F. A. (2019). *LoQuiero: Un caso práctico de Mobile-D* [Tesis de maestría, Universidad Politécnica de Madrid]. Repositorio Institucional UPM. <https://oa.upm.es/58690/>

Maraza Nina, J. M., & Rojas Huaripata, M. M. (2021). *Aplicativo móvil con realidad aumentada, utilizando la metodología Mobile – D, para la promoción de películas a los visitantes en los cines* (Tesis de pregrado). Universidad Autónoma del Perú. <https://repositorio.autonoma.edu.pe/handle/20.500.13067/1524>

Alayo Villanueva, A. A., & Pinedo Tenicela, G. R. (2022). *Aplicación móvil con realidad aumentada para la promoción de películas en cines utilizando la metodología Mobile-D* (Tesis de pregrado). Universidad Privada del Norte. <https://repositorio.upn.edu.pe/handle/11537/32024>

Sepa Mora, F. G. (2022). *Desarrollo de una aplicación móvil Android para la promoción y difusión de eventos y turismo del cantón Guaranda utilizando la metodología de desarrollo MOBILE-D* (Tesis de pregrado). Escuela Superior Politécnica de Chimborazo. <https://dspace.esPOCH.edu.ec/items/4b882b3b-6bad-4d19-b16f-c0e81c233ec6>

Castillo Romero, J. A. (2019). Big data. IFCT128PO. IC Editorial

Zúñiga, J. J. E. (2020). Aplicación de metodología CRISP-DM para segmentación geográfica de una base de datos pública. *Ingeniería Investigación Y Tecnología*, 21(1), 1–13. <https://doi.org/10.22201/fi.25940732e.2020.21n1.008>

Santana Falcón, F (2023). *Análisis de datos abiertos de fútbol aplicando la metodología CRISP-DM*. Tesis (Master), [E.T.S.I. Telecomunicación \(UPM\)](#).

Rodriguez, C. A. M., Pinzon, M. A. R., Quintero, L. M. P., & Vergel, L. M. A. (2024). Aplicación de machine learning y metodología CRISP-DM para la clasificación precisa de severidad en casos de dengue. *Revista Colombiana De Tecnologías De Avanzada (RCTA)*, 1(43), 78-85.

Sifuentes, M. S. G. C., Pérez, L. G. V., Cantabrana, M. G. N., Acosta, I. F. O., Santana, F. A. Á., & Fierro, M. D. L. Á. S. (2023). Modelo Predictivo de la Deserción Escolar en Educación Superior: una Aproximación desde la Minería de Datos Utilizando la Metodología CRISP-DM. *Ciencia Latina Revista Científica Multidisciplinar*, 7(5), 7797-7812.